



Data Cryptography Services (DCS) 3.7 Software Interface Specification (SIS)

Interface Specification/Control

Prepared By:

Electronic acknowledgment on file in Jira and wiki.

Michael J Pajevski Date
Author

Approved By:

Electronic signature on file.

Ken Gieselman Date
Title

Electronic signature on file.

Mike Pajevski Date
Title

Configuration Item: Data Cryptography Services – 681.2

Electronic signature on file.

MGSS Document No.: **DOC-005776 Rev A**
Issue Date: May 5, 2026

Richard Cho Date
Document Release Authority

Jet Propulsion Laboratory
California Institute of Technology

EPDM Electronic Signatures

User-Group/Role	...	Decision	Comments	Date
Gieselman, Ken G (kgieselm)-JPL Consumer/Proje...	...	Approve		31-Mar-2026 09:02
Pajevski, Michael J (pajevski)-JPL Consumer/Proje...	...	Approve		01-Apr-2026 15:35

Reviewers

Review solicited from the following and includes the required minimum reviewers (in bold) as specified in DOC-000016 Documentation Structure, Standards and Definitions. The Review Date identifies respondents but does not indicate approval or concurrence, only acknowledgement that the reviewer has reviewed the document, submitted recommendations/feedback, and are aware of the document and its impact.

Review Date	Name/Title
	Name (Bold if required) S/W Quality Assurance Engineer (SQAE)
	Ken G Gieselman SSE Application Security Subsystem (ASEC)
11/20/2025	Kyran J Owen-Mankovich MM Ground Data System Engineer (MGDSE)
3/13/2026	Michael J Pajevski AMMOS Architect (AA)
	Michele I Vogt Program Chief Engineer (PCE)
	Michele I Vogt AMMOS Element Manager (AEM) Data Cryptography Services (DCS)
3/13/2026	Michael J Pajevski Task Manager

Document Change Log

Revision	Issue Date	Affected Sections or Pages	Change Summary
Initial	09/11/2024	All	Initial release. Changes from DOC-005431 KMC 3.4 SIS: • Updated document references. • Updated all path references to new opensource delivery structure. • Added crypto-service/status API.
A	3/18/2026	All	Changed naming from Key Management and Cryptography (KMC) to Data Cryptography Services (DCS), particularly in Sections 1.1 (Identification), 1.2 (Overview), and 1.7 (Applicable and Referenced Documents). Marked DCS Java methods for the Java Crypto Service Client as deprecated in Sections 4.1.4.1.2 through 4.1.4.1.6., and mention of that change in Section 1.2. In Section 1.6, removed references to unpublished documentation and added the the DCS open-source repository URL. Added Section 1.7 to include an acknowledgement statement. Corrected filepaths to DCS software. Added AEAD to acronyms in Appendix A.

Table of Contents

Section 1 Introduction	1
1.1 Identification	1
1.2 Overview	1
1.3 Document Scope	1
1.4 Method	2
1.5 Terminology and Notation	2
1.6 Referenced Documents and Websites	2
Section 2 Interface Overview	3
2.1 Software Interface Overview	3
2.2 DCS Crypto Java Library Interface	4
2.3 DCS Crypto Service Interfaces	6
2.4 DCS Cryptography Command Line Interface	7
2.5 DCS SDLS Client Interfaces	7
2.6 DCS SDLS HTTPS API	7
2.7 DCS SA Management CLI	7
Section 3 Cryptography Interface Descriptions	8
3.1 DCS Cryptography Java Library Interface	8
3.2 DCS Crypto Service Interface	26
3.3 DCS Cryptography Command Line Interface	30
Section 4 Space Data Link Security (SDLS) Interfaces	34
4.1 DCS Client SDLS Interfaces	34
4.2 DCS SDLS HTTPS API Interfaces	50
4.3 Security Association Management Command Line Interface (CLI)	51
Section 5 DCS Secure Connection Interfaces	56
5.1 DCS Secure Connections	56
Appendix A: Acronyms and Abbreviations	58
Appendix B: DCS Metadata	60
Appendix C: Example Use of the DCS SDLS Python Interface	62

Section 1 Introduction

1.1 Identification

The Data Cryptography Sevrvices (DCS) (formerly Key Management and Cryptography (KMC)) software is an assembly in the Application Security (ASEC) subsystem within the Advanced Multi-Mission Operations System (AMMOS).

Table 1: Identification

Property	Value
Configuration Item (CI) #	681.2
Element	Systems Engineering Office (SEO)
Program Set	Data Cryptography Sevrvices (DCS)
Version	3.7.x

1.2 Overview

The DCS application programming interfaces (APIs) described in this document are used by software applications to perform cryptographic operations (e.g., encryption) on data.

The naming transition from KMC to DCS is in-progress. Documentation is being updated. The KMC term is used in references to previous documentation and previous software releases, and KMC terminology remains in the DCS software. The transition to DCS terminology will be completed in a series of future releases of DCS.

The latest DCS 3.7.x software release can be found at:

<https://github.com/NASA-AMMOS/DCS/releases/tag/3.7.1>

1.3 Document Scope

This document describes the software interfaces provided by the DCS version 3.7.x software. All of the data-level cryptography interfaces except the command line interface (CLI) existed in the 1.x releases. The DCS CLI was added in KMC 2.x. The Space Data Link Security (SDLS) C and Python interfaces were added in the 3.0.0 release, and the CLI for managing Security Associations (SAs) for SDLS was added in the 3.1.0 release. The SDLS HTTPS API was added in the 3.2.0 release. The Java Crypto Service Client (a part of DCS Java APIs that provides a pass-through to the DCS Crypto Service REST APIs) is marked as deprecated in this 3.7.x release and will be removed in the upcoming DCS 4.0.0 release.

This document provides an overview of the DCS software interfaces (see Section 2) and descriptions of the software interfaces provided by the DCS (see Section 3, 5, and 6).

This document does not cover software installation and configuration. See the DCS 3.7 Product Guide for instructions about deploying the software.

This document is effective upon its release date and is applicable to AMMOS subsystem development efforts as directed by MGSS management.

1.4 Method

The software interfaces described in this document are provided in the following forms:

1. Java Programming Language Interfaces
 - a. Java library (in .jar file format) that is integrated into software applications via Java application programming interfaces (APIs).
 - b. Only the data-level cryptography capabilities are provided via Java APIs.
2. Python Programming Language Interfaces
 - a. Software library that is integrated with software applications via Python APIs.
 - b. Only the SDLS capabilities are provided via Python APIs
3. C Programming Language Interfaces
 - a. Software library that is integrated with software applications via C APIs.
 - b. Only the SDLS capabilities are provided via C APIs
4. Web Service Interfaces
 - a. HTTPS APIs are provided for cryptography capabilities and for using SDLS with Telecommand (TC) transfer frames.
5. Command Line Interfaces
 - a. BASH scripts are provided for using data-level crypto capabilities.
 - b. An SDLS SA management CLI is provided to support command encryption.
6. Web Graphical User Interface (GUI)
 - a. Web GUI for managing SDLS SAs.
 - b. See the DCS 3.7 Product Guide for information about using the SA Management GUI.

1.5 Terminology and Notation

No special notation is used in this document. See Appendix A for acronym definitions.

1.6 Referenced Documents and Websites

Table 2: Referenced MGSS Documents

Document Title	Document ID
Data Cryptography Sevrvices (DCS) 3.7 Product Guide	DOC-005733

Table 3: Referenced Websites

Website Name	Website URL
DCS 3.7.x Software Repository	https://github.com/NASA-AMMOS/DCS/releases/tag/3.7.1

1.7 Acknowledgement

The research was carried out at the Jet Propulsion Laboratory, California Institute of Technology, under a contract with the National Aeronautics and Space Administration (80NM0018D0004).

Section 2

Interface Overview

2.1 Software Interface Overview

The software provides interfaces that software applications use to encrypt/decrypt digital information and/or create/verify integrity check values. The software provides those capabilities in a Java software library, as Web services, and via command line scripts.

supports the following algorithms:

1. Symmetric cryptography algorithms
 - a. AES encryption/decryption
 - i. 128-bit and 256-bit key lengths
 - ii. Cipher Block Chaining (CBC) mode
 - b. Triple-DES encryption/decryption
 - i. 168-bit key length
 - ii. Cipher Block Chaining (CBC) mode
 - c. Authenticated encryption/decryption
 - i. 128-bit and 256-bit key lengths
 - ii. AES with Galois/Counter Mode (GCM)
 - d. Create/verify SHA-1, SHA-256, SHA-384, and SHA-512 digests (a.k.a., hashes)
 - e. Create/verify SHA-1, SHA-256, SHA-384, and SHA-512 HMACs
 - f. Create/verify AESCMAC, DESedeCMAC CMACs
2. Asymmetric cryptography algorithms
 - a. RSA encryption/decryption
 - i. 2048-bit and 4096-bit key lengths
 - ii. Electronic Code Book (ECB) mode
 - b. RSA digital signature
 - i. Create/verify digital signatures with SHA1withRSA, SHA256withRSA, SHA384withRSA, SHA512withRSA algorithms

The also provides capabilities to apply and reverse the CCSDS Space Data Link Security (SDLS) protocol to telecommand (TC) transfer frames (TFs). Those capabilities are provided via Python, and C interfaces. The Python and C interfaces can be used to call local library functions for applying/reversing SDLS. An application uses the same Python or C APIs in either case.

See the DCS Product Guide for information about deploying and using the DCS software.

The DCS Crypto Client makes secure connections to the Key Management Service (KMS) and to the Crypto Service. The HTTPS APIs of the Crypto Service require secure connections. See Section 6 of this document for information about the secure connections used by the DCS.

The subsections below provide an overview of using the DCS Crypto APIs.

2.2 DCS Crypto Java Library Interface

The DCS software includes a Java library for performing cryptography. The DCS Crypto Java APIs can retrieve cryptographic keys from a Java keystore (in PKCS12 or BCFKS format) and/or (depending on how DCS is configured) from a Key Management Service (KMS) via the Key Management Interoperability Protocol (KMIP).

Only authorized endpoints (e.g., user workstation or server) can retrieve keys from a KMS. If a KMS is used to get keying material, the client host must have a TLS client certificate and private key in order to connect to the KMS. Access control for keying material is managed on the KMS.

2.2.1 Creating and Initializing a KmcCryptoManager

The first steps in using the DCS Crypto APIs are creating and initializing a KmcCryptoManager, which is done as follows:

1. A KmcCryptoManager is instantiated.
2. Software applications need a KmcCryptoManager to initialize the DCS software library and to create instances of the cryptography function objects that are used to encrypt data, decrypt data, create ICVs, and verify ICVs.
 - a. See Section 3.1.4.1 for a description of the KmcCryptoManager class.
 - b. The Constructor of the KmcCryptoManager configures it according to the parameters in DCS configuration files.
 - i. See Section 3.1.1 for a description of the DCS configuration files and how they control the operation of the DCS software libraries.
 - ii. Optionally, input arguments can be used to override configuration file parameters.
3. If a cipher transformation other than the default will be used to encrypt data, and/or an algorithm other than the default will be used to create ICVs, use the appropriate “set” method to select the desired transformation or algorithm. Selecting an algorithm or transformation other than the default is an advanced case, and is normally not necessary.
 - a. e.g., use setHmacAlgorithm(HmacSHA512) to create a SHA-512 based HMAC instead of using the default HmacSHA256 algorithm.

2.2.2 Encrypting Data

The KmcCryptoManager is used to create an Encrypter object, which is used to encrypt data (in the form of an InputStream). The steps below provide an overview of using the Encrypter:

1. The *createEncrypter* method of the KmcCryptoManager is used to create an Encrypter object (see Sections 3.1.4.4.1 and 3.1.4.4.2).
 - a. Input arguments passed to the createEncrypter method are used to select a specific key.
 - b. There are two forms of the createEncrypter method. One for getting keying material from either a KMS or a keystore according to kmc-crypto.cfg, and another for getting keying material from a keystore by specifying the keystore information. See Section 3.1.1 for details.
2. The *encrypt* method of the Encrypter is called to encrypt data (see Section 3.1.4.4.1 and 3.1.4.4.2).

- a. An `InputStream` and `OutputStream` are passed to the *encrypt* method as arguments.
- b. The return value of the *encrypt* method is a `String` that contains the encryption metadata (i.e., info about the key reference and transformation that were used), as defined in Appendix B.

The *encrypt* method may be used multiple times, as long as the same key is used to perform the encryption. If a different key will be used, create a different `Encrypter`.

2.2.3 Decrypting Data

The `KmcCryptoManager` is used to create a `Decrypter` object, which is used to decrypt cipher text (in the form of an `InputStream`). The steps below provide an overview of using the `Decrypter`:

1. The *createDecrypter* method of the `KmcCryptoManager` is used to create a `Decrypter` object (see Sections 3.1.4.1.12 and 3.1.4.1.13 of this document).
 - a. There are two forms of the *createDecrypter* method. One for getting keying material from either a KMS or a keystore according to `kmc-crypto.cfg` and another for getting keying material from a keystore by specifying the keystore information.
2. The *decrypt* method of the `Decrypter` is called to decrypt data (see Section 3.1.4.5.1 of this document).
 - a. The cipher text (as an `InputStream`) and encryption metadata (as a `String`) are passed into the method.
 - b. The plaintext is provided back to the application via an `OutputStream` that is given as an argument to the *decrypt* method.

The *decrypt* method may be used multiple times, as long as the same key location (i.e., KMS or keystore) is used. If a different key location will be used, create a different `Decrypter`.

2.2.4 Creating Integrity Check Values

The `KmcCryptoManager` is used to create an `IcvCreator` object, which is used to create ICVs. The steps below provide an overview of using the `IcvCreator`:

1. The *createIcvCreator* method of the `KmcCryptoManager` is used to create an `IcvCreator` object (see Sections 3.1.4.1.15, 3.1.4.1.16, and 3.1.4.1.16 of this document).
 - a. Arguments passed to the *createIcvCreator* method are used to select a particular key.
 - b. There are two forms of the *createIcvCreator* method for creating message digests (which do not involve the use of keying material), getting the keying material from a KMS, or getting keying material from a keystore.
2. The *createIntegrityCheckValue* method of the `IcvCreator` is called to create an ICV (see Section 3.1.4.6.1 of this document).
 - a. The input data (for the ICV) is passed into the method in the form of an `InputStream`.
 - b. The return value is a `String` containing the integrity metadata, as defined in Appendix B, which contains the ICV and references to the key and transformation that were used.

The *createIntegrityCheckValue* method may be used multiple times, as long as the same key is used. If a different key will be used, create a different `IcvCreator`.

2.2.5 Verifying Integrity Check Values

The KmcCryptoManager is used to create an IcvVerifier object, which is used to verify ICVs. The steps below provide an overview of using the IcvVerifier:

1. The *createIcvVerifier* method of the KmcCryptoManager is used to create an IcvVerifier object (see Sections 3.1.4.1.17 and 3.1.4.1.18 of this document).
 - a. There are two forms of the *createIcvVerifier* method for different sources of keying material (i.e., using a KMS vs. a keystore).
2. The *verifyIntegrityCheckValue* method of the IcvVerifier is called to verify an ICV (see Section 3.1.4.7.1 of this document).
 - a. The data to be verified is passed in as an InputStream and integrity metadata, as defined in Appendix B, is passed in as a String.
 - b. The return value is a Boolean that indicates if the input data matches the ICV (True) or not (False).

The *verifyIntegrityCheckValue* method may be used multiple times, as long as the same key location is used. If a different key location will be used, create a different IcvVerifier.

2.2.6 Handling Errors

The DCS Crypto Java APIs throw exceptions if there are errors. Exception classes provided by the DCS (i.e., KmcCryptoManagerException and KmcCryptoException) provide error codes and methods for getting the error codes. The method details in this SIS show which exceptions are thrown, and the descriptions of the exception classes describe the error codes and methods for getting them.

2.3 DCS Crypto Service Interfaces

DCS includes a Crypto Service that provides HTTPS interfaces for performing cryptographic operations on data similar to the DCS Java APIs. The DCS Crypto Service interfaces are documented in Section 3.2 of this document.

The URL of the Crypto Service interfaces accept parameters for the cryptographic operation, and the body of the request (i.e., the POST data) contains the data to be acted upon.

To encrypt data using the Crypto Service, the key reference (keyRef) is supplied as a URL parameter, and the plaintext to be encrypted is in the POST data. The response includes Encryption Metadata, as defined in Appendix B, and the ciphertext.

To decrypt data using the Crypto Service, the Encryption Metadata is supplied as a URL parameter, and the ciphertext is provided as POST data. The response includes the plaintext.

To create an ICV using the Crypto Service, the keyRef is supplied as a URL parameter and the data is provided as POST data. The response includes Integrity Metadata, as defined in Appendix B, which includes the ICV.

To verify an ICV, the Integrity Metadata is supplied as a URL parameter and the data to be verified is provided as POST data. The response includes a status of true if the data matches the ICV, or false if the data does not match the ICV.

2.4 DCS Cryptography Command Line Interface

The DCS software includes a set of CLI scripts, which utilize the Crypto Service. See Section 3.2.5 for descriptions of the CLI scripts.

There are four CLI tools, which are:

- `kmc-data-encrypt.sh`
- `kmc-data-decrypt.sh`
- `kmc-icv-create.sh`
- `kmc-icv-verify.sh`

2.5 DCS SDLS Client Interfaces

DCS includes Python, C, and REST programming language interfaces for applying the CCSDS SDLS protocol to Telecommand (TC) Transfer Frames (TFs), and for reversing SDLS.

DCS SDLS Python APIs (`apply_security_tc` and `process_security_tc`) are provided in Python 3.9 software libraries that are installed by the DCS Crypto Client package. The DCS SDLS Python APIs are wrappers around the DCS SDLS C software library. See Section 4.1.1 for a description of the DCS SDLS Python APIs.

The DCS SDLS C APIs (`apply_security_tc` and `process_security_tc`) are provided via a set of C software libraries, which include the open-source CryptoLib software from NASA IV&V (with major contributions from MGSS) and the DCS SDLS C API software library. The C libraries are installed by the DCS Crypto Client package. See Section 4.1.2 for a description of the DCS SDLS C APIs.

The processing performed by the DCS SDLS APIs is controlled by Security Associations (SAs). The DCS Product Guide provides SA management information and instructions.

2.6 DCS SDLS HTTPS API

The DCS SDLS Service includes an optional SDLS HTTPS API that can be configured to provide SDLS protocol operations to TC TFs. See Section 4.2 for details on endpoints and payloads.

2.7 DCS SA Management CLI

DCS Security Association Management Command Line Interface (CLI) provides a set of scripts that allow users and applications to use DCS security association commands : creating, updating, listing, deleting, keying or rekeying, starting, stopping or expiring a security association. These are discussed in detail in section 4.3.

Section 3

Cryptography Interface Descriptions

3.1 DCS Cryptography Java Library Interface

3.1.1 DCS Crypto Client Configuration files

The DCS Crypto Java Libraries use the following configuration files:

1. `kmc-crypto.cfg` – contains parameters used by the DCS Crypto Libraries.
The `kmc-crypto.cfg` configuration file is read by the `KmcCryptoManager` when it is instantiated. Applications can override configuration file parameters by using optional arguments in the constructor of the `KmcCryptoManager`.
2. `kmc-crypto-log4j2.xml` – contains configuration parameters for initializing log4j 2.x loggers used by the DCS Crypto Java APIs.

3.1.2 Location of the Configuration Files

The DCS software libraries expect to find the configuration file in the `/opt/ammos/kmc/etc` directory by default, but other locations can be used by either using the `-kmc_crypto_config_dir` argument when instantiating the `KmcCryptoManager` (see Section 3.1.4.1) or by including the configuration file directory in the `CLASSPATH` environment variable for the application.

3.1.3 Content of the Configuration Files

3.1.3.1 Content of the `kmc-crypto.cfg` File

The `kmc-crypto.cfg` configuration file uses the Java Properties file format. The file contains a set of parameters each of its key and value is a string.

The default configuration file has the following parameters as shown below. The configuration parameters that are commented out in the configuration file show the default values that are used in the code. Uncomment the parameter and put in a different value if necessary.

If `kmc-crypto.cfg` is configured for both a KMIP KMS and a keystore, DCS will request keys from the KMS if not found in the keystore.

```
#
# Configuration file for KMC Crypto Client
#

# Specify a KMS and/or Crypto Keystore to use as source(s) of keys.

# Keys are requested from the specified KMS if not found in the specified keystore.
#key_management_service_uri=tls12://fully-qualified-domain-name:5696

# crypto_keystore_type can be BCFKS, PKCS12, JCEKS, or JKS. FIPS should use BCFKS keystore.
# crypto_key_password is not used by PKCS12 keystore
#crypto_keystore_location=KEYSTORE_PATH
#crypto_keystore_type=PKCS12
#crypto_keystore_password=PASSWORD_OF_KEYSTORE
#crypto_key_password=PASSWORD_OF_ALL_KEYS
```

```

# false for the Crypto Java Client to use the Library, true to use the Service
use_crypto_service=false
crypto_service_uri=https://fully-qualified-domain-name:8443/crypto-service

# keystore for authenticating with KMS when TLS is used (FIPS should use .bcfks)
#keystore_file=/etc/pki/tls/private/ammos-server-keystore.bcfks
keystore_file=/etc/pki/tls/private/ammos-server-keystore.pl2
keystore_password=changeit

# parameters for connection to KMS
# default cipher suites: TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384, TLS_ECDHE_RSA_
WITH_AES_256_GCM_SHA384
#additional_cipher_suites="TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA384 TLS_ECDHE_RSA
_WITH_AES_256_CBC_SHA384"
connection_timeout_connect=5000
connection_timeout_read=30000

# key cache (time-to-live TTL in minutes)
key_cache_enable=true
key_cache_ttl=1440

# Crypto algorithms should use Java standard names:
# https://docs.oracle.com/en/java/javase/17/docs/specs/security/standard-names.h
tml

default_symmetric_encryption_algorithm=AES-256
allowed_symmetric_encryption_algorithms=AES-128:AES-256:DESede-168
default_asymmetric_encryption_algorithm=RSA-2048
allowed_asymmetric_encryption_algorithms=RSA-2048:RSA-3072:RSA-4096

default_message_digest_algorithm=SHA-256
allowed_message_digest_algorithms=SHA-1:SHA-256:SHA-384:SHA-512
default_hmac_algorithm=HmacSHA256
allowed_hmac_algorithms=HmacSHA1:HmacSHA256:HmacSHA384:HmacSHA512
default_cmac_algorithm=AESCMAC
allowed_cmac_algorithms=AESCMAC:DESedeCMAC
default_digital_signature_algorithm=SHA256withRSA
allowed_digital_signature_algorithms=SHA1withRSA:SHA256withRSA:SHA384withRSA:SHA
512withRSA

default_AES_transformation=AES/CBC/PKCS5Padding
allowed_AES_transformations=AES/CBC/PKCS5Padding:AES/GCM/NoPadding
default_DESede_transformation=DESede/CBC/PKCS5Padding
allowed_DESede_transformations=DESede/CBC/PKCS5Padding
default_RSA_transformation=RSA/ECB/OAEPWithSHA-256AndMGF1Padding
allowed_RSA_transformations=RSA/ECB/PKCS1Padding:RSA/ECB/OAEPWithSHA-256AndMGF1P
adding:RSA/ECB/OAEPWithSHA1AndMGF1Padding

# Default Providers
# https://docs.oracle.com/en/java/javase/17/security/oracle-providers.html
# Uncomment and change value if non-default provider is used.
#AES_provider=SunJCE
#DESede_provider=SunJCE
#RSA_provider=SunJCE
#SHA-1_provider=SUN
#SHA-256_provider=SUN
#SHA-384_provider=SUN
#SHA-512_provider=SUN
#HmacSHA1_provider=SunJCE
#HmacSHA256_provider=SunJCE
#HmacSHA384_provider=SunJCE
#HmacSHA512_provider=SunJCE

```

```
#SHA1withRSA_provider=SunRsaSign
#SHA256withRSA_provider=SunRsaSign
#SHA384withRSA_provider=SunRsaSign
#SHA512withRSA_provider=SunRsaSign

# Provider classes
#SUN=sun.security.provider.Sun
#SunJCE=com.sun.crypto.provider.SunJCE
BCFIPS_provider_class=org.bouncycastle.jcajce.provider.BouncyCastleFipsProvider

# Bouncy Castle FIPS provider
AES_provider=BCFIPS
AESCMAC_provider=BCFIPS
DESedeCMAC_provider=BCFIPS
AESGCM_provider=BCFIPS
```

3.1.3.2 Log4j Configuration Files

The DCS Crypto Java APIs use slf4j, and support log4j 2.x for logging. Support for log4j 1.2 was present in KMC 1.x and KMC 2.x, but discontinued in KMC/DCS 3.x. Applications that use DCS Crypto Java APIs use the DCS log4j configuration file to initialize DCS Crypto Java API log4j loggers. Below is the default location of the configuration file for log4j 2.x:

- log4j 2.x: /opt/ammos/kmc/etc/kmc-crypto-log4j2.xml

In KMC 2.x, applications use a Java property to specify location of the log4j configuration file. See Section 3.1.4 for information about the jar files that must be used for DCS logging in Java applications.

3.1.3.2.1 DCS Logging using Log4j 2.x

For log4j 2.x (a list of config files can be specified):

```
JAVA_OPTS="-Dlog4j.configurationFile=/opt/ammos/kmc/etc/kmc-crypto-
log4j2.xml,application-log4j2-config-file".
```

The `-Dlog4j.configurationFile` parameter will override any log4j configuration file loaded in the application code. For log4j 2.x, the application log4j configuration file should be added to the `-Dlog4j.configurationFile` property.

For Tomcat applications, the log4j configuration files should be added to the application's web.xml, as follows:

```
<context-param>
  <param-name>log4jConfiguration</param-name>
  <param-value>/opt/ammoskmc//etc/kmc-crypto-log4j2.xml,application-
log4j2-config-file</param-value>
</context-param>
```

3.1.3.2.2 Content of the kmc-crypto-log4j2.xml File

Log4j 2.x configuration files use an XML format. It defines logging properties, appender, and logger settings. Unless it is specified via `-kmc_crypto_config_dir` argument, its default location is /opt/ammos/kmc/etc/. Below is the default log4j 2.x configuration file.

```
<?xml version="1.0" encoding="UTF-8"?>
<Configuration status="info">

  <Properties>
    <Property name="kmcClientPath">/opt/ammos/kmc/logs</Property>
    <Property name="kmcClientDebug">kmc-crypto-client-debug</Property>
```

```

    <Property name="kmcClientAudit">kmc-crypto-client-audit</Property>
</Properties>

<Appenders>
  <Console name="LogToConsole" target="SYSTEM_OUT">
    <PatternLayout pattern="%m%n"/>
  </Console>
  <RollingFile name="LogKmcClientDebug" fileName="${kmcClientPath}/${kmcClient
Debug}.log"
    filePattern="${kmcClientPath}/${kmcClientDebug}-${d{yyMMdd_HHmms
s}.log.gz">
    <PatternLayout pattern "[%d{ISO8601}{UTC} UTC] [%t] %-5p %c - %m%n"/>
    <Policies>
      <SizeBasedTriggeringPolicy size="100MB"/>
    </Policies>
    <DefaultRolloverStrategy max="30"/>
  </RollingFile>
  <RollingFile name="LogKmcClientAudit" fileName="${kmcClientPath}/${kmcClient
Audit}.log"
    filePattern="${kmcClientPath}/${kmcClientAudit}-${d{yyMMdd_HHmms
s}.log.gz">
    <PatternLayout pattern "[%d{ISO8601}{UTC} UTC] %m%n"/>
    <Policies>
      <SizeBasedTriggeringPolicy size="100MB"/>
    </Policies>
    <DefaultRolloverStrategy max="30"/>
  </RollingFile>
</Appenders>

<Loggers>
  <Logger name="gov.nasa.jpl.ammos.kmc" level="info" additivity="false">
    <AppenderRef ref="LogKmcClientDebug"/>
  </Logger>
  <Logger name="ch.ntb.inf.kmip" level="info" additivity="false">
    <AppenderRef ref="LogKmcClientDebug"/>
  </Logger>
  <Logger name="AUDIT" level="info" additivity="false">
    <AppenderRef ref="LogKmcClientAudit"/>
  </Logger>
  <Root level="error">
    <AppenderRef ref="LogToConsole"/>
  </Root>
</Loggers>

</Configuration>

```

3.1.4 Java Programming Language Interfaces

DCS provides the Java Classes and Interfaces described in the following subsections.

See the DCS Software Design Document for a class diagram and other design information associated with the classes and interfaces described in the following subsections.

The DCS default deployment includes Java libraries in `/opt/ammos/kmc/lib`, but this path can be modified at build and/or deployment time for additional flexibility. Please make sure that any CLASSPATH settings reference the deployed location for *all* necessary libraries. The environment variable KMC_HOME can be defined at runtime to change the default DCS location from `/opt/ammos/kmc` to another location.

Applications that use the DCS Java APIs to perform cryptography with the local library functions provided by DCS must include the following in CLASSPATH:

- /opt/amos/kmc/lib/kmc-crypto.jar
- /opt/amos/kmc/lib/kmc-crypto-library.jar
- /opt/amos/kmc/lib/kmc-crypto.jar

The DCS Java APIs use gson for parsing JSON messages. Applications must include a gson jar file in CLASSPATH, as follows:

- gson-2.10.1.jar

The DCS Crypto Java APIs use slf4j and log4j for logging. Applications that use the DCS Crypto Java APIs must include the following slf4j and log4j packages in CLASSPATH.

- slf4j-api-2.0.12.jar
- log4j-slf4j-impl-2.23.1.jar
- log4j-api-2.23.1.jar
- log4j-core-2.23.1.jar

The DCS Crypto Java API log4j loggers are initialized using a configuration file, as described in Section 3.1.3.2.

The Bouncy Castle library is needed in all cases in order to use a FIPS 140 certified library.

- bc-fips-1.0.2.5.jar

The DCS Java APIs use of a Java truststore for SSL/TLS authentication with a Key Management Service (KMS) or DCS Crypto Service (i.e., when using the Crypto Client).

The DCS Java APIs search for a Java truststore in the following order:

1. If the Java system property "javax.net.ssl.trustStore" is set, it is used to find the truststore.
 - a. Note: If this property is set, the DCS Java APIs will not look at any other location.
 - i. Set the property by using the following command line argument when launching the Java application: -Djavax.net.ssl.trustStore=/path/to/amos-truststore.jks
 - ii. Or by using the following command in the Java application:
System.setProperty("javax.net.ssl.trustStore", "/path/to/amos-truststore.jks")
2. A file named .truststore in the user's home directory will be used if the file exists.
3. The following will be used if it exists: /etc/pki/tls/amos-truststore.jks
4. The default Java truststore will be used if none of the above is found (except when the Java system property is set as described in #1 and the truststore does not exist at that location).

3.1.4.1 Class KmcCryptoManager

```
gov.nasa.jpl.ammos.kmc.crypto
public class KmcCryptoManager
```

The KmcCryptoManager is the entry point for using the DCS Java APIs. It performs the following functions:

- Loads the configuration file for the parameters to be used by the crypto functions,
- Processes input arguments for setting values of the configuration parameters,
- Provides getter and setter methods for the configuration parameters,
- Is used to get implementation instances of the DCS Cryptography Classes, and
- Initializes the logger.

The default config directory is /opt/ammos/kmc/etc, but the application can use the

-kmc_crypto_config_dir=<path to configuration directory> argument in the KmcCryptoManager constructor if the directory is in a different location.

Many of the parameters in the kmc-crypto.cfg file specify the default and allowed cryptographic algorithms. These parameters cannot be set by input arguments, but the algorithm that gets used can be set by using the appropriate method (e.g., use the setHmacAlgorithm method to select a particular HMAC algorithm to use).

To select a different cipher transformation or algorithm to use when encrypting or creating an ICV, use the appropriate “set” method after creating the KmcCryptoManager instance. For example, use the setHmacAlgorithm method to choose a specific HMAC algorithm to use when creating ICVs.

The names of the algorithms follow the ones used in Java unless otherwise noted. An algorithm must be listed in the allowed algorithms list in order to use it. The value of the allowed algorithms is a list of names separated by a colon.

3.1.4.1.1 Constructor Detail

```
KmcCryptoManager(String[] args)
    throws KmcCryptoManagerException
```

The constructor creates a KmcCryptoManager instance and initializes it based on the parameters in the configuration file and input arguments. In most cases, input arguments are not used. The constructor accepts an array of arguments for overriding the default parameters in the configuration file. The following arguments can be used with this constructor:

1. -kmc_crypto_config_dir=<path to DCS configuration file directory>
2. -key_management_server_uri=<URI of the DCS Key Service>

Parameters:

args - input arguments

Throws:

KmcCryptoManagerException - if any error occurred during initialization

3.1.4.1.2 setUseCryptoService [DEPRECATED – will be removed in DCS 4.0.0]

```
public final void setUseCryptoService(String trueFalse)
```

Sets the flag whether to use Crypto Service (“true”) or Crypto Library (“false”).

3.1.4.1.3 getUseCryptoService [DEPRECATED – will be removed in DCS 4.0.0]

```
public final String getUseCryptoService()
```

Returns “true” if Crypto Service is used, “false” if Crypto Library is used.

3.1.4.1.4 getKmcCryptoServiceURI [DEPRECATED – will be removed in DCS 4.0.0]

```
public final String getKmcCryptoServiceURI()
```

Returns the URI of the DCS Crypto Service.

3.1.4.1.5 setSsoCookie [DEPRECATED – will be removed in DCS 4.0.0]

```
public final void setSsoCookie(String ssoCookie)
```

Sets the SSO Cookie that is obtained from the Common Access Manager (CAM) and is needed to access the DCS Crypto Service when it is set up to use CAM for access control.

Parameters:

ssoCookie – SSO Token cookie string has the format of ssoCookieName=ssoToken (cookie name can be obtained by calling CAM API SsoTokenManager.getSsoCookieName).

3.1.4.1.6 getSsoCookie [DEPRECATED – will be removed in DCS 4.0.0]

```
public final String getSsoCookie()
```

Returns the SSO cookie string in the form of ssoCookieName=ssoToken

3.1.4.1.7 setAlgorithmProvider

```
public final void setAlgorithmProvider(String algorithm,
                                     String provider)
```

Sets the provider for the specified crypto algorithm.

3.1.4.1.8 getAlgorithmProvider

```
public final String getAlgorithmProvider(String algorithm)
```

Returns the name of the provider for the specified crypto algorithm.

3.1.4.1.9 removeAlgorithmProvider

```
public final void removeAlgorithmProvider(String algorithm)
```

Removes the provider for the specified crypto algorithm so that the default provider is used.

3.1.4.1.10 createEncrypter – Configured for KMS and/or Keystore

```
public Encrypter createEncrypter(String keyRef)
                               throws KmcCryptoManagerException
```

Returns an instance of the Encrypter implementation class that is configured to obtain the keying material specified by keyRef from a Key Management Service or a keystore, according to the kmc-crypto.cfg settings. Depending on the setting the implementation class can be the Crypto Local Library for performing cryptographic functions locally or the Crypto Service Client that makes use of the DCS Crypto Service. The Encrypter can be used to perform symmetric and asymmetric encryption.

Parameters:

`keyRef` - A string identifying the key, also known as a key alias.

Returns:

An instance of the Encrypter implementation.

Throws:

`KmcCryptoManagerException`

3.1.4.1.11 createEncrypter – Configured for Keystore

```
public Encrypter createEncrypter(String keystoreLocation,
                                String keystorePass, String keystoreType,
                                String keyRef, String keyPass)
                                throws KmcCryptoManagerException
```

This interface is deprecated, and only provided for backward compatibility. For more consistent usage, please use the createEncrypter interface documented in Section 3.1.4.1.10.

Returns an instance of the Encrypter implementation class (Crypto Local Library implementation only) that is configured to obtain the keying material specified by `keyRef` from the specified keystore, overriding the KMS or keystore settings in `kmc-crypto.cfg`. A keystore password (`keystorePass`) and key password (`keyPass`) must be provided to get a key from a keystore that uses a keystore password and/or key password. When a keystore is used, the default cipher transformation is used unless the `setCipherTransformation` method of the `KmcCryptoManager` is used to select a different cipher transformation.

Parameters:

`keystoreLocation` - The path to the location of the keystore.

`keystorePass` - The password of the keystore. Can be null, if not set in the keystore.

`keystoreType` - The keystore type. Valid values are PKCS12, JKS, and KCEKS

`keyRef` - A string identifying the key, also known as alias.

`keyPass` - The password of the key to be retrieved. Can be null, if not set in the keystore.

Returns:

An instance of the Encrypter implementation.

Throws:

`KmcCryptoManagerException`

3.1.4.1.12 createDecrypter – Configured for KMS and/or Keystore

```
public Decrypter createDecrypter()
                                throws KmcCryptoManagerException
```

Returns an instance of the Decrypter implementation class configured to get keys from a KMS or a keystore, according to `kmc-crypto.cfg`. The Decrypter can be used to perform symmetric and asymmetric decryption.

Returns:

An instance of the Decrypter implementation.

Throws:

`KmcCryptoManagerException`

3.1.4.1.13 createDecrypter – Configured for Keystore

```
public Decrypter createDecrypter(String keystoreLocation,
                                String keystorePass, String keystoreType,
                                String keyPass) throws
                                KmcCryptoManagerException
```

This interface is deprecated, and only provided for backward compatibility. For more consistent usage, please use the createDecrypter interface documented in Section 3.1.4.1.12.

Returns an instance of the Decrypter implementation class that is configured to obtain the keying material from the specified keystore, overriding the KMS or keystore settings in crypto.cfg. A keystore password (keystorePass) and/or key password (keyPass) must be provided to get a key from a keystore that uses a keystore password and/or key password. When a keystore is used, the default cipher transformation is used unless the setCipherTransformation method of the KmcCryptoManager is used to select a different cipher transformation.

Parameters:

keystoreLocation - The path to the location of the keystore.
 keystorePass - The password of the keystore. Can be null, if not set in the keystore.
 keystoreType - The keystore type. Valid values include
 keyPass - The password of the key to be retrieved. Can be null, if not set in the keystore.

Returns:

An instance of the Encrypter implementation.

Throws:

KmcCryptoManagerException

3.1.4.1.14 createIcvCreator – Configured for Digests

```
public IcvCreator createIcvCreator()
                                throws KmcCryptoManagerException
```

Returns an instance of the IcvCreator implementation class that will be used to create message digests, and thus does not use any keying material.

Returns:

An instance of the IcvCreator implementation.

Throws:

KmcCryptoManagerException

3.1.4.1.15 createIcvCreator – Configured for KMS and/or Keystore

```
public IcvCreator createIcvCreator(String keyRef) throws
                                KmcCryptoManagerException
```

Returns an instance of the IcvCreator implementation class configured to get the keying material specified by keyRef from a KMS and/or a keystore, according to kmc-crypto.cfg. The IcvCreator is used to create Integrity Check Values (ICVs) for input data.

Parameters:

keyRef - A string identifying the key, also known as alias.

Returns:

An instance of the IcvCreator implementation.

Throws:

KmcCryptoManagerException

3.1.4.1.16 createIcvCreator – Configured for Keystore

```
public IcvCreator createIcvCreator(String keystoreLocation,
                                   String keystorePass, String keystoreType,
                                   String keyRef, String keyPass)
    throws KmcCryptomanagerException
```

This interface is deprecated, and only provided for backward compatibility. For more consistent usage, please use the createIcvCreator interface documented in Section 3.1.4.1.15.

Returns an instance of the IcvCreator implementation class that is configured to obtain the keying material specified by keyRef from the specified keystore overriding the KMS and/or keystore settings in kmc-crypto.cfg. A keystore password (keystorePass) and key password (keyPass) must be provided to get a key from a keystore that uses a keystore password and/or key password.

Parameters:

keystoreLocation - The path to the location of the keystore.
 keystorePass - The password of the keystore. Can be null, if not set in the keystore.
 keystoreType - The keystore type. Valid values that can be used include
 keyRef - A string identifying the key, also known as alias.
 keyPass - The password of the key to be retrieved. Can be null, if not set in the keystore.

Returns:

An instance of the Encrypter implementation.

Throws:

KmcCryptoManagerException

3.1.4.1.17 createIcvVerifier – Configured for Digests, KMS, and/or Keystore

```
public final IcvVerifier createIcvVerifier()
    throws KmcCryptoManagerException
```

Creates a IcvVerifier which does not need a cryptographic key (for example, message digest algorithms are used) or obtains the key from the a Key Management Service (KMS) or a keystore, according to kmc-crypto.cfg.

The metadata associated with the data to be verified provides all the information needed to perform the verification of the data. This metadata is defined in Appendix B.

Returns:

The IcvVerifier object.

Throws:

KmcCryptoManagerException

3.1.4.1.18 createIcvVerifier – Configured for Keystore

```
public IcvVerifier createIcvVerifier(String keystoreLocation,
                                   String keystorePass,
                                   String keystoreType, String keyPass)
    throws KmcCryptomanagerException
```

This interface is deprecated, and only provided for backward compatibility. For more consistent usage, please use the `createIcvVerifier` interface documented in Section 3.1.4.1.17.

Returns an instance of the `IcvVerifier` implementation class that is configured to obtain the keying material from the specified keystore, overriding the KMS or keystore settings in `crypto.cfg`. A keystore password (`keystorePass`) and key password (`keyPass`) must be provided to get a key from a keystore that uses a keystore password and/or key password.

Parameters:

`keystoreLocation` - The path to the location of the keystore.
`keystorePass` - The password of the keystore. Can be null, if not set in the keystore.
`keystoreType` - The keystore type. Valid values are PKCS12, JKS, and KCEKS
`keyPass` - The password of the key to be retrieved. Can be null, if not set in the keystore.

Returns:

An instance of the `Encrypter` implementation.

Throws:

`KmcCryptoManagerException`

3.1.4.1.19 `setMessageDigestAlgorithm`

```
public void setMessageDigestAlgorithm(String algorithm)
                                   throws KmcCryptoManagerException
```

Sets the algorithm to be used for creating a message digest with the `IcvCreator`. Use this method before creating an `IcvCreator` that will be used to create digests with an algorithm other than the default message digest algorithm specified in the `kmc-crypto.cfg` configuration file.

Parameters:

`algorithm` - the Message Digest algorithm to be used.

Throws:

`KmcCryptoManagerException` - if `algorithm` is invalid or not in the allowed list.

3.1.4.1.20 `getMessageDigestAlgorithm`

```
public final String getMessageDigestAlgorithm()
```

Returns the name of the currently set default message digest algorithm.

Returns:

The name of the default message digest algorithm.

3.1.4.1.21 `setHmacAlgorithm`

```
public void setHmacAlgorithm(String algorithm)
                                   throws KmcCryptoManagerException
```

Sets the algorithm that is used to create HMACs when a KMS is not used to get keys. The keying material retrieved from a KMS contains the name of the HMAC algorithm to be used. However, keying material retrieved from a keystore does not. In this case, this method can be used to override the default HMAC algorithm specified in the configuration file, before creating an `IcvCreator`.

Parameter:

`algorithm` - the algorithm to be used for creating HMACs if the keying material does not provide the name of the algorithm.

Throws:

`KmcCryptoManagerException` - if it is an invalid algorithm or not in the allowed list.

3.1.4.1.22 getHmacAlgorithm

```
public final String getHmacAlgorithm()
```

Returns:

The name of the default HMAC algorithm.

3.1.4.1.23 setCmacAlgorithm

```
public void setCmacAlgorithm(String algorithm)
                               throws KmcCryptoManagerException
```

Sets the algorithm to override the default CMAC algorithm. In the case of CMAC the CMAC algorithm is different from the key algorithm.

Parameters:

`algorithm` - the algorithm to be used for creating CMACs.

Throws:

`KmcCryptoManagerException` - if it is an invalid algorithm or not in the allowed list.

3.1.4.1.24 getCmacAlgorithm

```
public final String getCmacAlgorithm()
```

Returns the name of the default CMAC algorithm.

Returns:

The name of the default CMAC algorithm.

3.1.4.1.25 setDigitalSignatureAlgorithm

```
public void setDigitalSignatureAlgorithm(String algorithm)
                                           throws KmcCryptoManagerException
```

Sets the algorithm to override the default digital signature algorithm.

Parameter:

`algorithm` - the algorithm to be used for creating digital signatures.

Throws:

`KmcCryptoManagerException` - if it is an invalid algorithm or not in the allowed list.

3.1.4.1.26 getDigitalSignatureAlgorithm

```
public final String getDigitalSignatureAlgorithm()
```

Returns:

The name of the default digital signature algorithm.

3.1.4.1.27 setCipherTransformation

```
void setCipherTransformation(String transformation)
                               throws KmcCryptoManagerException
```

Sets the cipher transformation to be used for encryption using the algorithm specified in the transformation. Only used when it is necessary to override the default encryption transformation that is specified in the kmc-crypto.cfg configuration file. The input string should consist of algorithm/modeOfOperation/paddingScheme.

Parameter:

`transformation` – the cipher transformation to be used for performing encryption with the algorithm specified in the transformation.

Throws:

`KmcCryptoManagerException` – if transformation is invalid or not in the allowed list.

3.1.4.1.28 getCipherTransformation

```
public final String getCipherTransformation(String algorithm)
```

Returns the default cipher transformation for the specified encryption algorithm.

Parameter:

`algorithm` - The encryption algorithm.

Returns:

The default cipher transformation of the specified encryption algorithm.

3.1.4.1.29 setMacLength

```
void setMacLength(String macLength)
```

Sets the length of MACs to generate by an Encryptor (for AES-GCM) or IcvCreator, specifying the length as a String representing number of bits in the MAC.

Parameter:

`macLength` - Size (number of bits) of MAC to generate.

Returns:

The default cipher transformation of the specified encryption algorithm.

3.1.4.1.30 setMacLength

```
public final String setMacLength(int macLength)
```

Sets the length of MACs to generate by an Encryptor (for AES-GCM) or IcvCreator, specifying the length as an integer representing number of bits in the MAC.

Parameter:

`macLength` - Size (number of bits) of MAC to generate.

Returns:

The default cipher transformation of the specified encryption algorithm.

3.1.4.1.31 setMacLength

```
public final int getMacLength()
```

Gets the currently set value for the length (in bits) of MACs to generate with the Encryptor.

Returns:

Length (in bits) of MACs that will be generated by the Encryptor (for AES-GCM).

3.1.4.2 Class KmcCryptoManagerException

```
gov.nasa.jpl.ammos.kmc.crypto
```

```
public class KmcCryptoManagerException extends Exception
```

This exception is thrown by the DCS Crypto Manager to indicate that errors in the DCS config file and for any other reasons. Unknown parameters are ignored.

3.1.4.2.1 getErrorCode

```
public KmcCryptoManagerException.KmcCryptoManagerErrorCode  
getErrorCode()
```

Returns the error code of this exception.

Returns:

The error code associated with the instance of a KmcCryptoManagerException.

3.1.4.3 Enum KmcCryptoManagerException.KmcCryptoManagerErrorCode

```
gov.nasa.jpl.ammos.kmc.crypto
```

Enclosing class: KmcCryptoManagerException

```
public static enum KmcCryptoManagerException.KmcCryptoManagerErrorCode  
extends Enum<KmcCryptoManagerException.KmcCryptoManagerErrorCode>
```

Error code values for KmcCryptoManager exceptions are listed below.

Enum Constant Summary:

CONFIG_FILE_NOT_FOUND

The DCS config file (kmc-crypto.cfg) is not found.

CONFIG_PARAMETER_NOT_FOUND

An expected parameter is not found in the ASEC KMS config file.

CONFIG_PARAMETER_VALUE_INVALID

The value of a parameter in the config file or command-line argument is invalid.

CRYPTO_ALGORITHM_ERROR

Wrong cryptographic algorithm is used.

CRYPTO_KEY_ERROR

Failed to retrieve key or wrong key algorithm.

NOT_INITIALIZED

The DCS Crypto Manager has not been initialized.

NULL_VALUE

Unexpected null value.

3.1.4.4 Interface Encrypter

gov.nasa.jpl.ammos.kmc.crypto

public interface **Encrypter**

The Encrypter performs encryption using a key retrieved from a KMS or a keystore (see Section 4.1.3 for details). The key to be used, and the use of a KMS and/or keystore to get the key, is set when creating the Encrypter. The key determines the encryption algorithm, key length, and encryption type (symmetric or asymmetric) to be performed. The selection of the transformation (which includes the “mode of operation” and padding, in addition to the algorithm) is determined based on the default setting for the algorithm type (e.g., the default transformation for the AES algorithm is specified by the following entry in kmc-crypto.cfg: `default_transformation_AES=AES/CBC/PKCS5Padding`). The default transformation can be overridden by using the `setCipherTransformation` method in the `KmcCryptoManager` class before creating the Encrypter.

3.1.4.4.1 encrypt - Non-AEAD Algorithms

```
Public String encrypt(InputStream inputStream,
                      OutputStream outputStream)
                      throws KmcCryptoException
```

Encrypts the data read in from the input stream, outputs the cipher text to the output stream, and returns the encryption metadata, as defined in Appendix B, as a String.

Parameters:

`inputStream` – input stream attached to the data source to be encrypted. The `inputStream` will be closed upon completion.

`outputStream` – output stream attached to the data sink for encrypted data. The `outputStream` will be closed upon completion

Returns:

String containing the metadata to be used for decrypting the encrypted data.

Throws:

`KmcCryptoException`

3.1.4.4.2 encrypt - AEAD Algorithms

```
public String encrypt(InputStream inputStream, int encryptOffset
                      String iv, OutputStream outputStream)
                      throws KmcCryptoException
```

Encrypts the data read in from the input stream, outputs the cipher text to the output stream, and returns the encryption metadata, as defined in Appendix B, as a String.

Parameters:

`inputStream` - input stream attached to the data source to be encrypted. The `inputStream` will be closed upon completion.

`outputStream` - output stream attached to the data sink for encrypted data. The `outputStream` will be closed upon completion

`iv` - Initialization Vector. Use *null* to have DCS auto generate an IV. If supplied, the IV must be a URL-safe Base64 encoded String that is valid for the encryption algorithm (e.g., 128 bits for the AES algorithm). Do not re-use the same IV with the same encryption key (else the ciphertext is more vulnerable to cryptanalytic attacks).

`encryptOffset` - The byte from which encryption is applied. Only used for authenticated encryption.

Returns:

String containing the metadata, as defined in Appendix B, to be used for decrypting the encrypted data.

Throws:

`KmcCryptoException`

3.1.4.5 Interface Decrypter

`gov.nasa.jpl.amos.kmc.crypto`

`public interface Decrypter`

The Decrypter performs decryption of cipher text based on its associated metadata, as defined in Appendix B.

3.1.4.5.1 decrypt

```
public void decrypt(InputStream inputStream,
                    OutputStream outputStream,
                    String metadata)
    throws KmcCryptoException
```

Decrypts the encrypted data read from the input stream with the resulting plaintext data written to the output stream. The source (i.e., a KMS and/or keystore) of the key is selected when the Decrypter is created using the `KmcCryptoManager`. The key reference and other attributes needed to perform the decryption are contained in the encryption metadata, as defined in Appendix B, passed to the method.

Parameters:

`inputStream` - input stream attached to the data source to be decrypted. The `inputStream` will be closed upon completion.

`outputStream` - output stream attached to the data sink for decrypted data. The `outputStream` will be closed upon completion.

`metadata` - String containing the metadata to be used for decrypting the encrypted data. This parameter is defined in detail in Appendix B.

Throws:

`KmcCryptoException`

3.1.4.6 Interface IcvCreator

`gov.nasa.jpl.amos.kmc.crypto`

`public interface IcvCreator`

The IcvCreator creates an integrity check value (ICV) for input data. The resulting ICV, its associated key reference, and other crypto attributes are returned in a metadata string. The integrity of the data can be verified by the IcvVerifier using the metadata and input data. Metadata is defined in Appendix B.

3.1.4.6.1 createIntegrityCheckValue

```
public String createIntegrityCheckValue(InputStream inputStream)
                                throws KmcCryptoException
```

Returns the integrity check metadata for the input stream. The metadata contains the integrity check value, key reference, and other attributes that are used for ICV verification. These attributes are defined in Appendix B. KMC 3.1.0 added the ability to create truncated MACs by setting the macLength configuration parameter via the setMacLength(String macLength) method described in Section 3.1.4.1.29 or the setMacLength(int macLength) method described in Section 3.1.4.1.30.

Parameter:

`inputStream` - Input stream of the data for integrity check. The inputStream will be closed upon completion.

Returns:

String containing the integrity metadata.

Throws:

KmcCryptoException

3.1.4.7 Interface IcvVerifier

```
gov.nasa.jpl.amos.kmc.crypto
public interface IcvVerifier
```

The IcvVerifier checks the integrity of the input data based on its associated metadata. The metadata contains the integrity check value (ICV), key reference (if any), and crypto attributes that are used to create the ICV. These attributes are defined in Appendix B.

3.1.4.7.1 verifyIntegrityCheckValue

```
public boolean verifyIntegrityCheckValue(InputStream inputStream,
                                         String integrityMetadata)
                                throws KmcCryptoException
```

Verifies the input data matches the integrity check value.

Parameters:

`inputStream` - input stream for data to be verified against the integrity check metadata. The inputStream will be closed upon completion.

`integrityMetadata` - metadata containing integrity check value, its associated key reference, and crypto attributes used for ICV verification. Metadata is defined in Appendix B.

Returns:

True - if the input data matches the ICV.

False - if the input data does not match the ICV.

Throws:

`KmcCryptoException`

3.1.4.8 Class `KmcCryptoException`

`gov.nasa.jpl.ammos.kmc.crypto`

`public class KmcCryptoException extends Exception`

This exception is thrown by the DCS cryptography API for any error that occurs when performing cryptographic functions.

3.1.4.8.1 `getErrorCode`

`public KmcCryptoException.KmcCryptoErrorCode getErrorCode()`

Returns the error code for an instance of a `KmcCryptoException`.

Returns:

The error code associated with the instance of a `KmcCryptoException`.

3.1.4.9 Enum `KmcCryptoException.KmcCryptoErrorCode`

`gov.nasa.jpl.ammos.kmc.crypto`

Enclosing class: `KmcCryptoException`

`public static enum KmcCryptoException.KmcCryptoErrorCode extends Enum<KmcCryptoException.KmcCryptoErrorCode>`

Error code values for Crypto exceptions are listed below.

Enum Constant Summary:

`CRYPTO_ALGORITHM_ERROR`

Error occurred performing the crypto function.

`CRYPTO_KEY_ERROR`

Key not found, or key does not match crypto algorithm.

`CRYPTO_METADATA_ERROR`

Error occurred parsing the metadata.

`CRYPTO_MISC_ERROR`

Miscellaneous error while performing cryptography.

`INVALID_INPUT_VALUE`

Invalid input value.

`KMS_CONNECTION_ERROR`

Failed to connect to the key management service.

3.1.4.10 Class `KmcCryptoVersion`

`gov.nasa.jpl.ammos.kmc.crypto`

This Java program prints the DCS Crypto API version information and can be run by the following commands:

- 1) For the Crypto Interface: `java -jar kmc-crypto.jar.`
- 2) For the Crypto Local Library Implementation: `java -jar kmc-crypto-library.jar.`
- 3) For the Crypto Service Client Implementation: `java -jar kmc-crypto-client.jar.`

3.1.4.10.1 main

```
public static void main(String[] args)
```

The main program prints the DCS Crypto API version information.

Parameters:

`args` - command-line arguments are not used.

3.2 DCS Crypto Service Interface

The DCS software provides Web services for cryptography functions described in the following subsections. There are two ways to use the web services:

1. Java applications can use the DCS Java APIs to access the Crypto Service. The DCS APIs provide reusable code to access the Crypto Service.
2. Applications, especially those written in programming languages other than Java, can directly make requests to the web services and receive JSON responses. The web services can also be used directly by bash scripts using the curl command.

3.2.1 Encryption Web Service

The Encryption Web Service encrypts plaintext data with the specified key and other parameters and returns the cipher text and metadata.

URL:

- `/crypto-service/encrypt?keyRef=<keyRef>&<optional parameters>`

Method:

- POST

URL Parameters:

- Required Parameters:
 - `keyRef` – Key reference
- Optional Parameters:
 - `transformation` – Cipher transformation
 - `iv` - Initialization Vector. Optional. If supplied, the IV must be a URL-safe Base64 encoded String that is valid for the encryption algorithm (e.g., 128 bits for the AES). Do not re-use the same IV with the same encryption key (else the ciphertext is more vulnerable to cryptanalytic attack).
 - `encryptOffset` - Optional. The byte from which encryption is applied. Only used for authenticated encryption.

POST Data:

- Plaintext data of Content-Type: `application/octet-stream`

Status Codes:

- 200 OK – Request completed successfully
- 400 Bad Request – Invalid request parameter
- 500 Internal Server Error – Error in processing the request

Response JSON Object:

- Status: {statusCode, reason}
- Metadata: {metadataType, keyRef, keyLength, keyType, cryptoAlgorithm, cipherTransformation, initialVector, encryptOffset(optional)}
- Base64ciphertext

Example Request:

POST /crypto-service/encrypt?keyRef=kmc/test/AES256&iv= JmV9POnDBYJ1m5khMpSxug==
 Note: The POST data (i.e., request body) contains the plaintext to be encrypted.

Example Successful Response:

```
{
  "status": {
    "statusCode": 200,
    "reason": "OK"
  },
  "metadata": {
    "keyLength": 256,
    "keyRef": "kmc/test/AES256",
    "initialVector": "JmV9POnDBYJ1m5khMpSxug==",
    "cryptoAlgorithm": "AES",
    "cipherTransformation": "AES/CBC/PKCS5Padding",
    "metadataType": "EncryptionMetadata",
    "base64ciphertext": "Jf0wcMxtiDTLfQ7LwujF5R1Ir53sYrZGr7chGro71X6TFTqWwBtVQZelDpux4zwzHr4CSJknWBFCIoDtsPs/vQ=="
  }
}
```

Example Failed Response:

```
{
  "status": {
    "statusCode": 500,
    "reason": "EncryptService: Error in cipher transformation: gov.nasa.jpl.ammos.kmc.crypto.KmcCryptoManagerException: AES/CBC/NoPadding is not in the allowed list."
  }
}
```

3.2.2 Decryption Web Service

The Decryption Web Service decrypts ciphertext data with the specified metadata and returns the decrypted plaintext.

URL:

- /crypto-service/decrypt?metadata=<metadata>

Method:

- POST

URL Parameters:

- Required Parameters:
 - metadata – Metadata received from encryption, as defined in Appendix B

POST Data:

- Ciphertext data of Content-Type: application/octet-stream

Status Codes:

- 200 OK – Request completed successfully
- 400 Bad Request – Invalid request parameter
- 500 Internal Server Error – Error in processing the request

Response JSON Object:

- status: {statusCode, reason}
- base64cleartext: base64 encoded decryption output (i.e., clear text)

Example Request:

POST /crypto-service/decrypt?metadata=
keyLength:256,keyRef:kmc/test/AES256,initialVector:RICaJa/zKWw72DO/F5Y7iw==,crypto
Algorithm:AES,cipherTransformation:AES/CBC/PKCS5Padding,metadataType:EncryptionMet
adata

* Note: The POST data contains the ciphertext to be decrypted.

Example Successful Response:

```
{"status":{"httpCode":200,"reason":"OK"},"base64cleartext":"VGhpcyBpcyBhIHN0cmLuZyBmb3IgdGVzdGluZyB0aGUgS01DIGNyeXB0b2dyYXB0eS4K"}
```

Example Failed Response:

```
{"status":{"httpCode":500,"reason":"DecryptService: Exception during decryption:  
gov.nasa.jpl.ammos.kmc.crypto.KmcCryptoException: Exception on reading/writing io stream:  
java.io.IOException: javax.crypto.BadPaddingException: Given final block not properly  
padded"}}}
```

3.2.3 ICV Creation Web Service

The ICV Creation Web Service creates integrity check value of the input data with the specified key and other parameters. It returns the ICV and metadata, as defined in Appendix B.

URL:

- /crypto-service/icv-create?keyref=<keyRef>&<optional parameters>

Method:

- POST

URL Params:

- Required Parameters:
 - keyRef – Key reference
- Optional Parameters:
 - algorithm – The cryptographic algorithm for creating ICV. This parameter is only used with the Digest and Digital Signature algorithms.
 - macLength – Optional. Specifies a truncated MAC length.

Post Data:

- Input data of Content-Type: application/octet-stream

Status Codes:

- 200 OK – Request completed successfully
- 400 Bad Request – Invalid request parameter
- 500 Internal Server Error – Error in processing the request.

Response JSON Object:

- status: {httpCode, reason}
- metadata: {metadataType, keyRef, cryptoAlgorithm, integrityCheckValue}

Example Request:

POST /crypto-service/icv-create?keyRef=kmc/test/HmacSHA512

Note: The POST data contains the data that the ICV will be created for.

Example Successful Response:

```
{
  "status": {
    "httpCode": 200,
    "reason": "OK",
    "metadata": "integrityCheckValue:Gxulq7ZqWs05wGM1cB5j1wEPMcabWuNVYKALGL7HcvYbrdCg6yKE9AAfXBpOhtV9Tc1GCgwK0OKDrlF5GTWqIg==,keyRef:kmc/test/HmacSHA512,cryptoAlgorithm:HmacSHA512,metadataType:IntegrityCheckMetadata"
  }
}
```

Example Failed Response:

```
{
  "status": {
    "httpCode": 500,
    "reason": "IcvCreateService: Exception when creating IcvCreator: gov.nasa.jpl.ammos.kmc.crypto.KmcCryptoManagerException: digital signature algorithm HmacSHA512 is not in the allowed list."
  }
}
```

3.2.4 ICV Verification Web Service

The ICV Verification Web Service verifies the integrity of the input data with the specified metadata, as defined in Appendix B. The metadata, which contains the ICV, was generated by ICV Creation.

URL:

- `/crypto-service/icv-verify?metadata=<metadata>`

Method:

- POST

URL Params:

- Required Parameters:
 - `metadata` – Metadata generated by ICV Create
- Optional Parameters
 - No optional parameter for `icv-verify`. The `macLength`, if used, will be in the metadata.
 - `macLength` – specifies a truncated MAC length
- POST Data:
 - Input data of Content-Type: `application/octet-stream`

Status Codes:

- 200 OK – Request completed successfully
- 400 Bad Request – Invalid request parameter
- 500 Internal Server Error – Error in processing the request.

Response JSON Object:

- Status: `{httpCode, reason}`
- Result: `true` or `false`

Example Request:

```
POST /crypto-service/icv-verify?metadata=integrityCheckValue:Gxulq7ZqWs05wGM1cB5j1wEPMcabWuNVYKALGL7HcvYbrdCg6yKE9AAfXBpOhtV9Tc1GCgwK0OKDrlF5GTWqIg==,keyRef:kmc/test/HmacSHA512,cryptoAlgorithm:HmacSHA512,metadataType:IntegrityCheckMetadata
```

Note: The POST data contains the data to be verified.

Example Successful Response:

```
{
  "status": {
    "httpCode": 200,
    "reason": "OK",
    "result": true
  }
}
```

Example Failed Response:

```
{ "status": { "httpCode": 400, "reason": "IcvVerifyService: Exception during ICV verification: gov.nasa.jpl.amos.kmc.crypto.KmcCryptoException: Missing crypto algorithm in metadata." } }
```

3.2.5 Crypto Service Status

URL:

- /crypto-service/status

Method:

- Get

URL Params: None

Status Codes:

- 200 OK – Request completed successfully
- 400 Bad Request – Invalid request parameter
- 500 Internal Server Error – Error in processing the request.

Response JSON Object:

- Status: {httpCode, reason}

Example Request:

```
GET /crypto-service/status
```

Example Successful Response:

```
{ "httpCode": 200, "reason": "The KMC Crypto Service is up and running." }
```

3.3 DCS Cryptography Command Line Interface

DCS Command Line Interface (CLI) provides a set of scripts that allow users and applications to use DCS crypto service: data encryption and decryption, data integrity check value creation and verification. Below is the set of DCS CLI API calls:

1. kmc-data-encrypt.sh
2. kmc-data-decrypt.sh
3. kmc-icv-create.sh
4. kmc-icv-verify.sh

Note: In order to use DCS CLI, users and scripts/applications are required to obtain an SSO token from a CAM Server beforehand. The same SSO token can be reused until the token expires. When the SSO token expires, obtain a new SSO token from a CAM Server before using the DCS CLI scripts again.

3.3.1 DCS Encryption CLI (kmc-data-encrypt.sh)

```
kmc-data-encrypt.sh [-v] -keyRef=KEY_REFERENCE -input=FILE_PATH
                    -output=FILE_PATH -metadata=FILE_PATH [-transform=TRANSFORMATION]
                    [-cam_cookie_file=FILE_PATH] [-crypto_service_uri=URI]
                    [-iv=INITIALIZATION_VECTOR] [encryptOffset=INTEGER]
```

where:

-v	Verbose mode. By default, verbose mode is OFF. To enable verbose mode, the verbose option -v must be entered as the 1 st argument.
-keyRef	Reference to the key be used for data encryption
-input	Path to the file to be encrypted (i.e., the plaintext)
-output	Path to output file (i.e., where to write the ciphertext).
-metadata	Encryption metadata output file path. Generated after data encryption is complete and used for decryption.
-transform	Transformation type for data encryption (optional). Supported transformation types are: AES/CBC/PKCS5Padding, AES/GCM/NoPadding, DESede/CBC/PKCS5Padding, RSA/ECB/PKCS1Padding, RSA/ECB/OAEPWithSHA-256AndMGF1Padding, RSA/ECB/OAEPWithSHA1AndMGF1Padding If not specified, the default transformation will be used.
-cam_cookie_file	Path to the cookie file that contains the SSO Token. This optional argument is needed only when the cookie filename is different than the default cookie file ~/.cam_cookie_file
-crypto_service_uri	Crypto Service URL (e.g., https://crypto123.example.com:8443/kmc-crypto-service). If not specified, the crypto_service_uri setting in /opt/ammos/kmc/etc/kmc-crypto.cfg will be used.
-iv	Initialization Vector. Optional. If supplied, the IV must be a URL-safe Base64 encoded String that is valid for the algorithm (e.g., 128 bits for the AES). Do not re-use the same IV with the same encryption key (else ciphertext is more vulnerable to cryptanalytic attacks).
encryptOffset	The byte from which encryption is applied. Only used for authenticated encryption.

Return codes:

- 0 Success
- 1 Error

Example #1:

```
$ kmc-data-encrypt.sh -keyRef="kmc/test/AES128" -input="sample.txt" \
  -output="encSample.dat" -metadata="encSample.meta" \
  -transform="AES/CBC/PKCS5Padding"
```

Example #2:

```
$ kmc-data-encrypt.sh -keyRef="kmc/test/AES128" -input=sample.txt \
  -output=encSample.dat -metadata=encSample.meta \
  -transform="AES/CBC/PKCS5Padding" -cam_cookie_file=~/.mycookiefile \
  -crypto_service_uri=https://asec-dev-vm33.jpl.nasa.gov:8443/crypto-service
```

3.3.2 DCS Decryption CLI (kmc-data-decrypt.sh)

```
kmc-data-decrypt.sh [-v] -input=FILE_PATH -output=FILE_PATH -metadata=FILE_PATH
[-cam_cookie_file=FILE_PATH] [-crypto_service_uri=URI]
```

where:

-v	Verbose mode. By default, verbose mode is OFF. To enable verbose mode, the verbose option -v must be entered as the 1 st argument.
-input	Path to ciphertext file to be decrypted.
-output	Path to output file (i.e., where to write the plaintext).
-metadata	Path to encryption metadata file.
-cam_cookie_file	Pathname to the cookie file that contains the SSO Token. This optional argument is needed only when the cookie filename is different than the default cookie filename ~/.cam_cookie_file
-crypto_service_uri	Crypto Service URL (e.g., https://crypto123.example.com:8443/kmc-crypto-service). If not specified, the crypto_service_uri setting in /opt/ammos/kmc/etc/kmc-crypto.cfg will be used.

Return codes:

0	Success
1	Error

Example #1:

```
$ kmc-data-decrypt.sh -cam_cookie_file=~/.mycookiefile \
  -metadata=encSample.meta -input=encSample.dat -output=decryptedSample.txt
```

Example #2:

```
$ kmc-data-decrypt.sh -input=encSample.dat -output=decryptedSample.txt \
  -metadata=encSample.meta -transform="AES/CBC/PKCS5Padding" \
  -cam_cookie_file=~/.mycookiefile \
  -crypto_service_uri=https://kmc.example.com:8443/crypto-service
```

3.3.3 DCS ICV Creation CLI (kmc-icv-create.sh)

```
kmc-icv-create.sh [-v] -keyRef=KEY_REFERENCE -input=FILE_PATH
-metadadata= FILE_PATH [-algorithm=ICV_ALGORITHM]
[-cam_cookie_file=FILE_PATH] [-crypto_service_uri=URL]
```

where:

-v	Verbose mode. By default, verbose mode is OFF. To enable verbose mode, the verbose option -v must be entered as the 1 st argument.
-keyRef	Reference to the key to be used for generating the ICV.
-algorithm	Needed for Message Digest and Digital Signature, and not used for other algorithms (which are determined by the key information). Supported algorithms are listed in Section 4.1.3.1.
-input	Data file to be used for generating the ICV file
-metadadata	Integrity metadata output filename containing the ICV and parameters needed to verify it

- cam_cookie_file Pathname to the cookie file that contains the SSO Token. This optional argument is only needed when the cookie filename is different than the default cookie filename ~/.cam_cookie_file
- crypto_service_uri Crypto Service URL (e.g., <https://crypto123.example.com:8443/kmc-crypto-service>). If not specified, crypto_service_uri setting specified in /opt/ammos/kmc/etc/kmc-crypto.cfg will be used.

Return codes:

- 0 Success (ICV created successfully)
- 1 Error (Error in ICV creation)

Example #1: Creating a Message Authentication Code (MAC):

```
$ kmc-icv-create.sh -KeyRef="kmc/test/AES128" -input=sample.txt \
  -metadata=sample.icv
```

Example #2: Creating a Digital Signature

```
$ kmc-icv-create.sh -cam_cookie_file=~/.mycookiefile \
  -input= sample.txt -metadata=sample.icv \
  -algorithm=SHA256withRSA
```

Example #3: Creating a Message Digest (i.e., no key)

```
$ kmc-icv-create.sh -input= sample.txt -metadata=sample.icv \
  -algorithm=SHA-256
```

3.3.4 DCS ICV Verification CLI (kmc-icv-verify.sh)

```
kmc-icv-verify.sh [-v] -input=FILE_PATH -metadata= FILE_PATH
  [-cam_cookie_file= FILE_PATH] [-crypto_service_uri=URL]
```

where:

- v Verbose mode. By default, verbose mode is OFF. Note, to enable verbose mode, the verbose option -v must be entered as the 1st argument.
- input File associated with the ICV (i.e., file to be integrity checked).
- metadata Integrity metadata file that is generated by kmc-icv-create.sh CLI.
- cam_cookie_file Pathname to the cookie file that contains the SSO Token. This optional argument is needed only when the cookie filename is different than the default cookie filename ~/.cam_cookie_file
- crypto_service_uri Crypto Service URL (e.g., <https://crypto123.example.com:8443/kmc-crypto-service>). If not specified, the crypto_service_uri setting in /opt/ammos/kmc/etc/kmc-crypto.cfg will be used.

Return codes:

- 0 Success (ICV is verified true)
- 1 Error (Error in verifying ICV)
- 2 INVALID_ICV (ICV is verified false)

Example:

```
$ kmc-icv-verify.sh -input= sample.txt -metadata=sample.icv \
  -cam_cookie_file=~/.mycookiefile
```

Section 4

Space Data Link Security (SDLS) Interfaces

4.1 DCS Client SDLS Interfaces

The DCS Crypto Client package includes Python and C software library APIs for applying and removing SDLS to/from TC TFs. The next two subsections describe the DCS SDLS Python & C APIs.

4.1.1 DCS SDLS Python Interfaces

The DCS SDLS Python APIs are provided at the following default locations:

- /opt/ammos/kmc/lib/Python3.9

The DCS SDLS Python APIs can be used by instantiating an `KmcSdlsClient` (which includes passing it configuration parameters) and then calling the `ApplySecurityTC` method (passing it a TC TF to apply SDLS to) and/or the `ProcessSecurityTC` method (passing it a TC TF that SDLS will be removed from).

4.1.1.1 The `KmcSdlsClient.py` Class

The `SdlsClient.py` class provides methods for initializing and using command encryption.

4.1.1.1.1 DCS Python `KmcSdlsClient` Constructor

The `KmcSdlsClient` Constructor is called to construct a `KmcSdlsClient` object that can be used to apply SDLS to TC TFs (via `ApplySecurityTC` method) and remove SDLS from TC TFs (via the `ProcessSecurityTC` method). The constructor takes a List of attribute-value pair configuration parameters.

The `KmcSdlsClient` constructor has the following form:

```
KmcSdlsClient(List config)
```

Parameters:

`config` – a list of attribute-value pairs of configuration parameters. See the list of valid parameters below.

Returns:

`KmcSdlsClient` – an instance of `KmcSdlsClient`

Configuration Parameters:

```
# The following indicates which type of SA DB to use.
cryptolib.sadb.type=inmemory|mariadb
# Default is mariadb.

# The following indicates the fully-qualified domain name (FQDN) of the MariaDB server.
cryptolib.sadb.mariadb.fqdn=<FQDN>
# For example: cryptolib.sadb.mariadb.fqdn=mdb.example.com
# Default is "localhost". Must be an actual FQDN if TLS is used.

# The following indicates the TCP port number of the MariaDB server.
cryptolib.sadb.mariadb.port=<port>
# Default: 3306
# The following indicates the database name in MariaDB.
cryptolib.sadb.mariadb.database_name=<db-name>
```

```

# Default:  sadb (recommended value)

# For all forms of authentication, use the following parameter:
cryptolib.sadb.mariadb.username=<username>
# Has default of sadb_user, but should be set to project-specific username.

# For password authentication (not recommended), use the following parameter:
cryptolib.sadb.mariadb.password=<password>
# Default is "", and is set to something else only if using password authentication.

# Setting the following to true prevents the client from making insecure connections.
cryptolib.sadb.mariadb.require_secure_transport=<true|false>
# Default is "false", but recommend setting to "true"

# The following indicates if the MariaDB server should be authenticated.
cryptolib.sadb.mariadb.tls.verifyserver=true|false
# Default is "false", and should be set to true if require_secure_transport is true.

# The following is used for tls & mtls connections. It indicates the CA Bundle to use.
cryptolib.sadb.mariadb.tls.cacert=<path/to/ca-bundle>
# Default is /etc/pki/tls/certs/amos-ca-bundle.crt

# The following is used for tls & mtls connections. It indicates the CA path.
cryptolib.sadb.mariadb.tls.capath=<path/to/ca-cert-directory>
# Default is "", and does not need to be changed (unless using a directory of CA certs
# instead of a CA cert bundle file specified by the previous parameter).

# The following indicates the client-side certificate (in .pem format) for mTLS connections.
cryptolib.sadb.mariadb.mtls.clientcert=<path/to/cert-file>
# For example: cryptolib.sadb.mariadb.mtls.clientcert=$HOME/sdls_client_cert.pem
# Default is ""

# The following indicates the client-side key (in .pem format) for mTLS connections.
cryptolib.sadb.mariadb.mtls.clientkey=<path/to/key-file>
# For example: cryptolib.sadb.mariadb.mtls.clientkey=$HOME/sdls_client_key.pem
# Default is ""

# The following indicates password for client key. Default is an empty string (for no password).
cryptolib.sadb.mariadb.mtls.clientkeypassword=<key passphrase, if used>
# Default is ""

# The following indicates which crypto capability to use.
cryptolib.crypto.type=libgcrypt|kmccryptoservice
# Default is "kmccryptoservice"

# The following indicates the connection protocol to the KMC Crypto Service.
cryptolib.crypto.kmccryptoservice.protocol=<libcurl protocol>
# Default is "https"

# The following indicates the FQDN of the KMC Crypto Service.
cryptolib.crypto.kmccryptoservice.fqdn=<FQDN>
# Default is "localhost". Must be changed to FQDN if TLS is used.

# The following indicates the TCP port of the KMC Crypto Service.
cryptolib.crypto.kmccryptoservice.port=<port>
# Default is 8443

# The following indicates the application endpoint of the crypto service.
cryptolib.crypto.kmccryptoservice.app=<KMC Crypto Service application context>
# Default is "crypto-service", which does not need to be changed.

# The following is always used. It indicates the CA Bundle to use.
cryptolib.crypto.kmccryptoservice.cacert=<path/to/ca-bundle>
# Default is /etc/pki/tls/certs/amos-ca-bundle.crt

# The following is optional. It indicates the CA Bundle Path to use. This is not necessary if
the cacert property above links directly to the absolute path of the cacert bundle.
cryptolib.crypto.kmccryptoservice.cacertpath=<path/to/ca-cert-directory>

```



```

# Default is "", and does not need to be changed (unless using a directory of CA certs
# instead of a CA cert bundle file specified by the previous parameter).

# The following is optional. It indicates the Issuer Cert to use. This is not always necessary
for TLS or mTLS connections.
cryptolib.crypto.kmccryptoservice.issuercert=<path/to/issuer-cert>
# Default is ""

# The following indicates if the KMC Crypto Service should be authenticated.
cryptolib.crypto.kmccryptoservice.verifyserver=true|false
# Default is "true"

# The following indicates the client-side certificate for mTLS connections.
cryptolib.crypto.kmccryptoservice.mtls.clientcert=<path/to/cert-file>
# For example: cryptolib.crypto.kmccryptoservice.mtls.clientcert=/etc/pki/tls/certs/client-cert.pem
# Default is ""

# The following indicates the client-side certificate format for mTLS connections. "PEM|P12|DER"
cryptolib.crypto.kmccryptoservice.mtls.clientcertformat=<cert format>
# Default is "PEM"

# The following indicates the client-side key (in .pem format) for mTLS connections.
cryptolib.crypto.kmccryptoservice.mtls.clientkey=<path/to/key-file>
# For example: cryptolib.crypto.kmccryptoservice.mtls.clientkey=/etc/pki/tls/private/client-key.pem
# Default is ""

# The following indicates password for client key.
cryptolib.crypto.kmccryptoservice.mtls.clientkeypassword=<key passphrase>
# Default is "" (for no password).

# The following controls the behavior that happens when a rollover event occurs in a truncated
MAC scenario.
cryptolib.tc.on_rollover_increment_nontransmitted_counter=<true/false>
#Default is "true"

# The following indicates if the ProcessSecurityTC capability should ignore anti-replay.
cryptolib.process_tc.ignore_antireplay=true|false
# The default is "true".

# The following indicates if the ProcessSecurityTC capability should ignore SA state.
cryptolib.process_tc.ignore_sa_state= true|false
# The default is "true".

The following indicates if the PDUs (i.e., for Extended Procedures) will be processed.
cryptolib.process_tc.process_pdus= true|false
# The default is "false".

The following indicates the VCID bitmask.
cryptolib.tc.vcid_bitmask=<bitmask>
# For example: cryptolib.tc.vcid_bitmask=0b000111 (for JPL)
# The default is "0b111111".

The following indicates if the frame has a PUS header.
cryptolib.tc.has_pus_header=true|false
# The default is "false".

The following indicates if an Error Correction Field (ECF) should be created.
cryptolib.apply_tc.create_ecf=true|false
# The default is "false".

```

The following indicates if the Error Correction Field (ECF) should be checked by CryptoLib as valid. If true and the calculated ECF checksum doesn't match the ECF in the processed frame, CryptoLib will return an error code.

```
cryptolib.process_tc.check_ecf=true|false
# The default is "false".
```

The following indicates if there is a unique Security Association for each MapID (Only relevant when dealing with segmentation headers). If false, uplink streams with only use GVCID when mapping uplink packets to Security Associations.

```
cryptolib.tc.unique_sa_per_mapid=true|false
# The default is "false".
```

The following indicates whether CAM security is enabled for the KMC Crypto Service that the client libraries communicate with. If true, CAM cookies are submitted with every encrypt and decrypt function call.

```
cryptolib.cam.enabled=false|true
# The default is false
```

The following indicates the CAM login method for generating the CAM cookies. "NONE" means that no CAM login occurs automatically in KMC, and a valid preauthenticated CAM SSO Token cookie must be already available in the CAM cookie file (see cryptolib.cam.cookie_file). "KERBEROS" means that Kerberos will be used automatically by KMC to generate CAM cookies (if they are not found in the cookie file), the user must have an active Kerberos TGT. "KEYTAB_FILE" means that a keytab file will be used automatically by KMC for Kerberos login and to generate the necessary CAM cookies (if they are not found in the cookie file).

```
cryptolib.cam.login_method=NONE|KERBEROS|KEYTAB_FILE
# The default is "NONE"
```

The following is used to specify the path to the CAM cookie file with the cookies ready to be transmitted to the KMC Crypto Service for authentication.

```
cryptolib.cam.cookie_file=/path/to/cookie_file
# The default is $HOME/.cam_cookie_file
```

The following is used to specify the path to the Kerberos keytab file used to login the user and generate new CAM cookies when the current set expires.

```
cryptolib.cam.keytab_file=/path/to/keytab_file
# There is no default for this configuration parameter.
```

The following is used to specify the URI for the CAM server needed to generate new CAM cookies when KERBEROS or KEYTAB_FILE login methods are used. This is not a full URL (i.e., it ends with the port number, and does not include /cam at the end).

```
cryptolib.cam.access_manager_uri=https://proj-caml.example.com:443
# There is no default for this configuration parameter.
```

The following is used to specify the username when KEYTAB_FILE type authentication is used to login and generate new CAM cookies.

```
cryptolib.cam.username=cryptosvc
# There is no default for this configuration parameter.
```

The following indicates the CAM home directory for the installed CAM Client software.

```
cryptolib.cam.cam_home=/ammos/css
# The default is "/ammos/css"
```

The following 4 properties refer to managed parameters for a given gvcid. They will need to be set for each GVCID that you intend to uplink with and apply or process security.

The following indicates if the frames will have an ECF.

```
cryptolib.tc.<scid>.<vcid>.<tfvn>.has_ecf=true|false
# The default is "false".
```

The following indicates the maximum transfer frame length.

```
cryptolib.tc.<scid>.<vcid>.<tfvn>.max_frame_length=<integer>
# The default is 1024.
```

The following indicates if the frame has a segmentation header.

```
cryptolib.tc.<scid>.<vcid>.<tfvn>.has_segmentation_header=true|false
# The default is "false".
```

For example, a set of managed parameters that apply to SCID 44, VCID 1, TFVN 0 might look as follows:

```
['cryptolib.tc.44.1.0.has_segmentation_header=false',
'cryptolib.tc.44.1.0.has_ecf=true',
'cryptolib.tc.44.1.0.max_frame_length=1024']
```

4.1.1.1.2 DCS Python apply_security_tc Method

The `apply_security_tc` method applies the CCSDS SDLS protocol to TC TFs. The API accepts a TC TF with or without an ECF. The configuration parameters (see the previous subsection) must be set correctly to indicate the presence or lack of the ECF.

This method has the following form:

```
bytearray apply_security_tc(bytearray TcTransferFrame)
                                throws SdlsClientException
```

Parameters:

`TcTransferFrame` - a TC TF without an ECF

Returns:

A bytearray containing a TC TF with SDLS

4.1.1.1.3 DCS Python process_security_tc Method

The `processSecurityTC` API removes the CCSDS SDLS protocol from TC TFs. The `processSecurityTC` API accepts a TC TF with SDLS applied, and returns the TC TF with SDLS removed.

This method has the following form:

```
TC process_security_tc(bytearray sdlsTcTransferFrame)
                                throws SdlsClientException
```

Parameters:

`sdlsTcTransferFrame` - a bytearray containing a TC TF with SDLS, and without an ECF

Returns:

TC - a named tuple collection containing a breakdown of the TC Transfer Frame, including the header, authentication trailer, and frame data (see section 5.1.1.2 for details).

4.1.1.2 ProcessSecurityTcOutput Class

The TC Class includes the results of the process_security_tc method.

```
class TC(NamedTuple):
    tc_header: TC_FramePrimaryHeader
    tc_security_header: TC_FrameSecurityHeader
    tc_pdu: bytearray
    tc_security_trailer: TC_FrameSecurityTrailer
class TC_FramePrimaryHeader(NamedTuple):
    tfvn: int      # Transfer Frame Version Number
    bypass: int    # Bypass Flag
    cc: int        # Command Control Flag
    spare: int     # Spare Bits
    scid: int      # Spacecraft ID
    vcid: int      # Virtual Channel ID
    fl: int        # Frame Length
    fsn: int       # Frame Sequence Number
class TC_FrameSecurityHeader(NamedTuple):
    sh: int        # Segment Header
    spi: int       # Security Parameter Index
    iv: bytearray  # Initialization Vector
class TC_FrameSecurityTrailer(NamedTuple):
    mac: bytearray # Message Authentication Code
    fecf: int      #Frame Error Control Field
```

4.1.2 DCS SdlsClientException Class

class SdlsClientException(Exception)

This class defines the exceptions that can be raised by the KmcSdlsClient.

4.1.2.1 get_error_code

```
get_error_code(self)
```

Returns one of the following error codes:

```
NO_FRAME_DATA = "NO_FRAME_DATA"
BAD_DATA_FORMAT = "BAD_DATA_TYPE"
INVALID_CONNECTION_TYPE = "INVALID_CONNECTION_TYPE"
INVALID_CONFIGURATION = "INVALID_CONFIGURATION"
SDLS_INITIALIZATION_ERROR = "SDLS_INITIALIZATION_ERROR"
APPLY_SECURITY_EXCEPTION = "APPLY_SECURITY_EXCEPTION"
PROCESS_SECURITY_EXCEPTION = "PROCESS_SECURITY_EXCEPTION"
INVALID_CONFIGURATION_VALUE = "INVALID_CONFIGURATION_VALUE"
MISSING_CONFIGURATION_PARAMETER = "MISSING_CONFIGURATION_PARAMETER"
FILE_DOESNT_EXIST = "FILE_DOESNT_EXIST"
INVALID_MANAGED_PARAMETER_FORMAT="INVALID_MANAGED_PARAMETER_FORMAT"
```

For CryptoLib error codes and their enum strings, see section 4.1.3.8.

4.1.3 SDLS C Interfaces

The subsections below describe the SDLS C APIs provided by DCS.

4.1.3.1 DCS `sdl_init` Function

The `sdl_init()` function is used to initialize the C library for SDLS. It is always the first function called when using the library.

This function has the following form:

```
int32_t sdl_init(void)
```

Returns:

An int32 representing the status code. See Section 4.1.3.8 for error code definitions.

4.1.3.2 DCS Configuration Functions

The functions in the subsections below must be called, before calling `apply_security_tc` or `process_security_tc` functions, to configure the command encryption capabilities in the CryptoLib library used by DCS.

4.1.3.2.1 DCS `sdl_config_cryptolib` Function

`sdl_config_cryptolib` is the primary configuration function for CryptoLib. Most importantly, it configures whether CryptoLib will use mariadb or an inmemory SADB, as well as the DCS Crypto Service or libgcrypt for cryptographic functions. It has the following signature:

```
int32_t sdl_config_cryptolib(uint8_t sadb_type, uint8_t
                             cryptography_type, uint8_t
                             crypto_create_fecf, uint8_t
                             process_sdl_pdu, uint8_t has_pus_hdr,
                             uint8_t ignore_sa_state, uint8_t
                             ignore_anti_replay, uint8_t
                             unique_sa_per_mapid, uint8_t
                             crypto_check_fecf, uint8_t vcid_bitmask,
                             uint8_t crypto_increment_nontransmitted_iv)
```

Parameters:

`sadb_type` - indicates the type of SA DB. Must be either:

`SADB_TYPE_INMEMORY` (for inmemory SA DB)

`SADB_TYPE_MARIADB` (for MariaDB SA DB)

`cryptography_type` - indicates the crypto mechanism to use. Must be either:

`CRYPTOGRAPHY_TYPE_LIBGCRYPT` (to use libgcrypt -- for dev/test only, not production)

`CRYPTOGRAPHY_TYPE_KMCCRYPTO` (to use the DCS Crypto Service)

`crypto_create_fecf` - indicates whether or not CryptoLib is expected to calculate the Frame Error Control Field (FECF) and return TCs with the FECF. Must be either:

`CRYPTO_TC_CREATE_FECF_FALSE` (do not create FECF)

`CRYPTO_TC_CREATE_FECF_TRUE` (do create FECF)

`process_sdls_pdus` - indicates whether or not to process SDLS extended procedure Protocol Data Units (PDUs) in CryptoLib. Must be either:

`TC_PROCESS_SDLS_PDUS_FALSE` (do not process PDUS)
`TC_PROCESS_SDLS_PDUS_TRUE` (do process PDUS)

`has_pus_hdr` - indicates the presence of the Primary Utilization Standard Header (typically used by ESA missions). Must be either:

`TC_NO_PUS_HDR` (PUS header is present)
`TC_HAS_PUS_HDR` (PUS header is not present)

`ignore_sa_state` - indicates whether or not to ignore the state of the Security Association (SA) in CryptoLib. Must be either:

`TC_IGNORE_SA_STATE_FALSE` (Do not ignore the state of the SA)
`TC_IGNORE_SA_STATE_TRUE` (Ignore the state of the SA)

`ignore_anti_replay` - indicates whether or not the `process_security_tc` function should ignore the anti-replay counter. Must be either:

`TC_IGNORE_ANTI_REPLAY_FALSE` (Do not ignore anti-replay)
`TC_IGNORE_ANTI_REPLAY_TRUE` (Ignore anti-replay)

`unique_sa_per_mapid` - indicates whether or not there is a unique Security Association (SA) for each Map ID. Must be either:

`TC_UNIQUE_SA_PER_MAP_ID_FALSE` (There is no unique SA per map ID)
`TC_UNIQUE_SA_PER_MAP_ID_TRUE` (There is a unique SA per map ID)

`crypto_check_fecf` - indicates whether or not the `process_security_tc` function should check the FECF in the input TC. Must be either:

`TC_CHECK_FECF_FALSE` (FECF is not present)
`TC_CHECK_FECF_TRUE` (FECF is present)

`Vcid_bitmask` - the bitmask for the Virtual Channel ID

`crypto_increment_nontransmitted_iv` - indicates whether or not the nontransmitted initialization vector should be incremented. The values must be either:

`SA_INCREMENT_NONTRANSMITTED_IV_FALSE` (Should not be incremented)
`SA_INCREMENT_NONTRANSMITTED_IV_TRUE` (should be incremented)

An example usage of this function is as follows:

```
sdls_config_cryptolib(SADB_TYPE_MARIADB, CRYPTOGRAPHY_TYPE_KMCCRYPTO,
                      CRYPTO_TC_CREATE_FECF_TRUE,
                      TC_PROCESS_SDLS_PDUS_FALSE,
                      TC_NO_PUS_HDR, TC_IGNORE_SA_STATE_FALSE,
                      TC_IGNORE_ANTI_REPLAY_TRUE,
                      TC_UNIQUE_SA_PER_MAP_ID_FALSE, SA_INCREMENT_
                      T_NONTRANSMITTED_IV_FALSE
                      TC_CHECK_FECF_TRUE, 0x3F);
```

Returns:

An int32 representing the status code. See Section 4.1.3.8 for error code definitions.

4.1.3.2.2 DCS sdls_config_mariadb Function

`sdls_config_mariadb` is a configuration function to configure the SDLS connection to MariaDB when CryptoLib is configured to use MariaDB. Its function signature is as follows:

```
int32_t sdls_config_mariadb(char* mysql_hostname, char* mysql_database,
                           uint16_t mysql_port, uint8_t
                           mysql_require_secure_transport, uint8_t
                           mysql_tls_verify_server, char*
                           mysql_tls_ca, char* mysql_tls_capath,
                           char* mysql_mtls_cert, char*
                           mysql_mtls_key, char*
                           mysql_mtls_client_key_password, char*
                           mysql_username, char* mysql_password);
```

Parameters:

`mysql_hostname` – the hostname for the connection

`mysql_database` – the database associated with the connection (usually `sadb`)

`mysql_port` – the port associated with the connection (usually 3306)

`mysql_require_secure_transport` – whether or not the client will require a secure (i.e., TLS) connection to the MariaDB SA DB (0 for false, or 1 for true)

`mysql_tls_verify_server` – indicates if the MariaDB server should be authenticated in the TLS connection

`mysql_tls_ca` – the Certificate Authority (CA) certificate bundle to use to verify connections to the DCS Crypto Service (e.g., `/etc/pki/tls/certs/ammos-ca-bundle.crt`)

`mysql_tls_capath` – the path to a directory of CA certs to use instead of a CA bundle. Can be set to NULL, unless this parameter is used instead of `kmc_tls_ca_bundle`.

`mtls_cert` – the certificate for the mTLS connections (usually “PEM”)

`mysql_mtls_key` – the key for mTLS connections (usually “PEM”)

`mtls_client_key_password` – the client key password for the mTLS client (can be NULL)

`mysql_username` – the username (can be NULL)

`mysql_password` – the password (can be NULL)

Note: Using mTLS (not username/password authentication) is recommended.

An example usage of this function is as follows:

```
sdls_config_mariadb ("mdb.example.com", "sadb", 3306, 1, 1,
                    "/etc/pki/tls/certs/ammos-ca-bundle.crt",
                    NULL, "/etc/pki/tls/certs/client-
                    cert.pem", "/etc/pki/tls/private/client-
                    key.pem", NULL, NULL, NULL);
```

Returns:

An int32 representing the status code. See Section 4.1.3.8 for error code definitions.

4.1.3.2.3 DCS sdls_config_kmc_crypto_service Function

This method configures the connection from CryptoLib to the DCS Crypto Service when CryptoLib is configured to use the DCS Crypto Service (as opposed to using libgcrypt). The function signature is as follows:

```
int32_t sdls_config_kmc_crypto_service(char *protocol, char
                                     *kmc_crypto_hostname, uint16_t
                                     kmc_crypto_port, char *kmc_crypto_app,
                                     char *kmc_tls_ca_bundle, char
                                     *kmc_tls_ca_path, uint8_t
                                     kmc_ignore_ssl_hostname_validation, char
                                     *mtls_client_cert_path, char
                                     *mtls_client_cert_type, char
                                     *mtls_client_key_path, char
                                     *mtls_client_key_pass, char
                                     *mtls_issuer_cert);
```

Parameters:

`Protocol` – the connection protocol (use "https")

`kmc_crypto_hostname` – fully qualified domain name (e.g., `kmc.example.com`) of the DCS Crypto Service server

`kmc_crypto_port` – connection port on DCS Crypto Service (usually 8443)

`kmc_crypto_app` – DCS Crypto Service application name (use "kmc-crypto-service")

`kmc_tls_ca_bundle` – the Certificate Authority (CA) certificate bundle to use to verify connections to the DCS Crypto Service (e.g., `/etc/pki/tls/certs/ammos-ca-bundle.crt`)

`kmc_tls_ca_path` – path to a directory of CA certs to use instead of a CA bundle. Can be set to NULL, unless this parameter is used instead of `kmc_tls_ca_bundle`.

`kmc_ignore_ssl_hostname_validation` – whether or not to ignore the ssl hostname validation (set to 1 for true, or 0 for false)

`mtls_client_cert_type` – the certificate type for mTLS (usually "PEM")

`mtls_client_key_path` – the client key path for the mTLS client

`mtls_client_key_pass` – the client key password for the mTLS client (can be NULL)

`mtls_issuer_cert` – the issuer for the mTLS certificate (can be NULL)

An example usage of this function is:

```
sdls_config_kmc_crypto_service("https", "kmc.example.com", 8443,
                              "crypto-service",
                              "/etc/pki/tls/certs/ammos-ca-bundle.crt",
                              NULL, 0, "/etc/pki/tls/certs/client-
                              cert.pem",
                              "PEM", "/etc/pki/tls/certs/client-
                              key.pem", NULL, NULL);
```

Returns:

An int32 representing the status code. See Section 4.1.3.8 for error code definitions.

4.1.3.2.4 DCS sdls_config_cam Function

This method configures the CAM behavior for the connection from CryptoLib to the DCS Crypto Service when CryptoLib is configured to use the DCS Crypto Service. The function signature is as follows:

```
int32_t sdls_config_cam(uint8_t cam_enabled, char* cookie_file_path,
                        char* keytab_file_path, uint8_t
                        login_method, char* access_manager_uri,
                        char* username, char* cam_home);
```

Parameters:

`cam_enabled` - controls whether CAM logic is enabled or disabled when CryptoLib encrypts and decrypts transfer frames via the DCS Crypto Service

`cookie_file_path` - path to the CAM cookie file

`keytab_file_path` - The path to a valid Kerberos Keytab File. Necessary for CAM_LOGIN_KEYTAB_FILE login_method only. Else, can be null.

`login_method` - Enum for the login_method to use with CAM. Options are: "CAM_LOGIN_NONE, CAM_LOGIN_KERBEROS, CAM_LOGIN_KEYTAB_FILE"

`access_manager_uri` -URI of the CAM server, but without the /cam context (e.g., "https://proj-cam1.example.com:8443")

`username` -username for logging into CAM

`cam_home` -root directory for the installed CAM Client (normally "/ammos/css")

Example usage:

```
sdls_config_cam(CAM_ENABLED_TRUE, "/home/testuser/.cam_cookie_file",
"/home/testuser/.kt/testuser333.kt",CAM_LOGIN_NONE,"https://somehost.example.com:443",
"testuser3300","/ammos/css")
```

Returns:

An int32 representing the status code. See Section 4.1.3.8 for error code definitions.

4.1.3.2.5 DCS sdls_config_add_gvcid_managed_parameter Function

`sdls_config_add_gvcid_managed_parameter` is used to configure the necessary managed parameters for each GVCID that you intend to apply and process security on. The function signature is as follows:

```
int32_t sdls_config_add_gvcid_managed_parameter(uint8_t tfvn, uint16_t
scid, uint8_t vcid, uint8_t has_fecf,
uint8_t has_segmentation_hdr, uint16_t
max_tc_frame_size );
```

Parameters:

`tfvn` - Transfer Frame Version Number

`scid` - Spacecraft ID

`vcid` - Virtual Channel ID

`has_fecf` - whether or not a Frame Error Control Field (FECF) is present

`has_segmentation_hdr` - whether or not the Telecommand Segment Header is present

`max_tc_frame_size` - maximum allowable size for the frame

Example usages of this function for multiple GVCIDs is as follows:

```
sdls_config_add_gvcid_managed_parameter(0, 0x002C, 0, TC_HAS_FECF,
                                         TC_NO_SEGMENT_HDRS);

sdls_config_add_gvcid_managed_parameter(0, 0x002C, 1, TC_HAS_FECF,
                                         TC_NO_SEGMENT_HDRS);
```

Returns:

An int32 representing the status code. See Section 4.1.3.8 for error code definitions.

4.1.3.3 DCS apply_security_tc Function

The apply_security_tc API applies the CCSDS SDLS protocol to TC TFs. The apply_security_tc API accepts a TC TF and returns the TC TF with SDLS applied. Optionally, the TC TF can be returned with an ECF.

This method has the following form:

```
int32_t apply_security_tc (const uint8_t* p_in_frame, const uint16_t
                          in_frame_length, uint8_t **pp_enc_frame,
                          uint16_t *p_enc_frame_len)
```

Parameters:

p_in_frame - a pointer to the input TC TF without SDLS
in_frame_length - the length of the input frame
pp_enc_frame - a pointer to the output frame that will have the SDLS applied
p_enc_frame_len - a pointer to the length of the output frame

Returns:

An int32 representing the status code. See Section 4.1.3.8 for error code definitions.

4.1.3.4 DCS apply_security_tc_cam Function

Same as apply_security_tc function above, but CAM cookies are also supplied to the Crypto Service when CAM is enabled via the sdls_config_cam function.

This method has the following form:

```
int32_t apply_security_tc (const uint8_t* p_in_frame, const uint16_t
                          in_frame_length, uint8_t **pp_enc_frame,
                          uint16_t *p_enc_frame_len, char*
                          cam_cookies)
```

Parameters:

p_in_frame - a pointer to the input TC TF without SDLS
in_frame_length - the length of the input frame
pp_enc_frame - a pointer to the output frame that will have the SDLS applied
p_enc_frame_len - a pointer to the length of the output frame
cam_cookies - a string of valid cam_cookies to use

Returns:

An int32 representing the status code. See Section 4.1.3.8 for error code definitions.

4.1.3.5 DCS process_security_tc Function

The process_security_tc API removes the CCSDS SDLS protocol from TC TFs. The process_security_tc API accepts a TC TF with SDLS applied and without an ECF, along with a pointer to relevant arguments (such as an optional Boolean “ignoreAntiReplay” parameter, which is false by default, in order to ignore anti-replay counters, and returns the TC TF with SDLS removed.

This method has the following form:

```
int32 process_security_tc (char* sdls_transfer_frame, int
                        len_sdls_frame, TC_t*
                        tc_sdls_processed_frame)
```

Parameters:

sdls_transfer_frame - input TC TF with SDLS applied

len_sdls_frame - length of the input sdls_transfer_frame character array

tc_sdls_processed_frame - structure of results of process_security_tc, which contains the following (from crypto_structs.h):

```
typedef struct
{
    TC_FramePrimaryHeader_t    tc_header;
    TC_FrameSecurityHeader_t   tc_sec_header;
    uint8                      tc_pdu[TC_FRAME_DATA_SIZE];
    uint16                     tc_pdu_len;
    TC_FrameSecurityTrailer_t  tc_sec_trailer;
} TC_t;

typedef struct
{
    uint8  tfvn    :2;           // Transfer Frame Version Number
    uint8  bypass  :1;           // Bypass
                                   // 0 = Type A: Sequence Check,
                                   // 1 = Type B: Checks are bypassed
    uint8  cc      :1;           // Control Command
                                   // 0 = Type D: Transfer Frame is Data
                                   // 1 = Type C: Control Command (for
                                   // COP)
    uint8  spare   :2;           // Reserved Spare - Shall be 00
    uint16 scid    :10;          // Spacecraft ID
                                   // Master Channel ID (MCID) = TFVN + SCID
    uint8  vcid    :6;           // Virtual Channel ID
    uint16 fl      :10;          // The whole transfer frame length (max
                                   // 1024)
    uint8  fsn     :8;           // Frame sequence number, also N(S), zeroed
                                   // on Type-B frames
```

```

} TC_FramePrimaryHeader_t;
typedef struct
{
    uint8    sh:TC_SH_SIZE;           // Segment Header
    uint16 spi;                       // Security Parameter Index
    uint8    iv[IV_SIZE];             // Initialization Vector for
                                     encryption
    //uint8    sn[TC_SN_SIZE];        // Sequence Number for anti-replay
    //uint8    pad[TC_PAD_SIZE];      // Count of the used fill Bytes
} TC_FrameSecurityHeader_t;
typedef struct
{
    uint8    mac[MAC_SIZE];           // Message Authentication Code
    uint16 fecf;                      // Frame Error Control Field
} TC_FrameSecurityTrailer_t;

```

Returns:

An int32 representing the status code. See Section 4.1.3.8 for error code definitions.

4.1.3.6 DCS process_security_tc_cam Function

Same as process_security_tc function above, but CAM cookies are also supplied to the Crypto Service when CAM is enabled via the sdls_config_cam function.

This method has the following form:

```

int32 process_security_tc (char* sdls_transfer_frame, int
                          len_sdls_frame, TC_t*
                          tc_sdls_processed_frame, char*
                          cam_cookies)

```

Parameters:

sdls_transfer_frame - input TC TF with SDLS applied
len_sdls_frame - length of the input sdls_transfer_frame character array
tc_sdls_processed_frame - structure of results of process_security_tc, which contains the following (from crypto_structs.h):
cam_cookies - a string with valid cam_cookies to use

Returns:

An int32 representing the status code. See Section 4.1.3.8 for error code definitions.

4.1.3.7 DCS sdls_get_error_code_enum_string Function

This function takes a CryptoLib error code as a parameter and returns an enum String of that error code.

This method has the following form:

```
char* sdls_get_error_code_enum_string(int32_t crypto_error_code)
```

Parameters:

`crypto_error_code` – The CryptoLib error code

Returns:

A `char*` that holds an enumeration of the error code.

4.1.3.8 DCS Error Codes

All of the error codes returned by DCS & CryptoLib functions are available in the `crypto_error.h` header file in the NASA CryptoLib github repository. As of 1.0.2, the error codes are defined as follows.

```
#define CRYPTO_CONFIGURATION_NOT_COMPLETE 100
#define CRYPTO_MANAGED_PARAM_CONFIGURATION_NOT_COMPLETE 101
#define CRYPTO_MARIADB_CONFIGURATION_NOT_COMPLETE 102
#define MANAGED_PARAMETERS_FOR_GVCID_NOT_FOUND 103

#define SADB_INVALID_SADB_TYPE 200
#define SADB_NULL_SA_USED 201

#define SADB_MARIADB_CONNECTION_FAILED 300
#define SADB_QUERY_FAILED 301
#define SADB_QUERY_EMPTY_RESULTS 302
#define SADB_INSERT_FAILED 303

#define CRYPTOGRAPHY_INVALID_CRYPTO_INTERFACE_TYPE 400
#define CRYPTOGRAPHY_UNSUPPORTED_OPERATION_FOR_KEY_RING 401
#define CRYPTOGRAPHY_LIBRARY_INITIALIZATION_ERROR 402

#define CRYPTOGRAPHY_KMC_CRYPTO_SERVICE_CONFIGURATION_NOT_COMPLETE 500
#define CRYPTOGRAPHY_KMC_CURL_INITIALIZATION_FAILURE 501
#define CRYPTOGRAPHY_KMC_CRYPTO_SERVICE_CONNECTION_ERROR 502
#define CRYPTOGRAPHY_KMC_CRYPTO_SERVICE_AEAD_ENCRYPT_ERROR 503
#define CRYPTOGRAPHY_KMC_CRYPTO_SERVICE_AEAD_DECRYPT_ERROR 504
#define CRYPTOGRAPHY_KMC_CRYPTO_JSON_PARSE_ERROR 505
#define CRYPTOGRAPHY_KMC_CIPHER_TEXT_NOT_FOUND_IN_JSON_RESPONSE 506
#define CRYPTOGRAPHY_KMC_CRYPTO_SERVICE_GENERIC_FAILURE 507
#define CRYPTOGRAPHY_KMC_CRYPTO_SERVICE_AUTHENTICATION_ERROR 508
#define CRYPTOGRAPHY_KMC_CRYPTO_SERVICE_MAC_VALIDATION_ERROR 509
#define CRYPTOGRAPHY_KMC_ICV_NOT_FOUND_IN_JSON_RESPONSE 510
#define CRYPTOGRAPHY_KMC_NULL_ENCRYPTION_KEY_REFERENCE_IN_SA 511
#define CRYPTOGRAPHY_KMC_NULL_AUTHENTICATION_KEY_REFERENCE_IN_SA 512
#define CRYPTOGRAPHY_KMC_CRYPTO_SERVICE_EMPTY_RESPONSE 513
#define CRYPTOGRAPHY_KMC_CRYPTO_SERVICE_DECRYPT_ERROR 514
#define CRYPTOGRAPHY_KMC_CRYPTO_SERVICE_ENCRYPT_ERROR 515
```

```

#define CAM_CONFIG_NOT_SUPPORTED_ERROR 600
#define CAM_INVALID_COOKIE_FILE_CONFIGURATION_NULL 601
#define CAM_AUTHENTICATION_FAILURE_REDIRECT 602

#define CRYPTO_LIB_SUCCESS (0)
#define CRYPTO_LIB_ERROR (-1)
#define CRYPTO_LIB_ERR_NO_INIT (-2)
#define CRYPTO_LIB_ERR_INVALID_TFVN (-3)
#define CRYPTO_LIB_ERR_INVALID_SCID (-4)
#define CRYPTO_LIB_ERR_INVALID_VCID (-5)
#define CRYPTO_LIB_ERR_INVALID_MAPID (-6)
#define CRYPTO_LIB_ERR_INVALID_CC_FLAG (-7)
#define CRYPTO_LIB_ERR_NO_OPERATIONAL_SA (-8)
#define CRYPTO_LIB_ERR_NULL_BUFFER (-9)
#define CRYPTO_LIB_ERR_UT_BYTE_MISMATCH (-10)
#define CRYPTO_LIB_ERR_NO_CONFIG (-11)
#define CRYPTO_LIB_ERR_INVALID_FECF (-12)
#define CRYPTO_LIB_ERR_BAD_ANTIREPLAY_WINDOW (-13)
#define CRYPTO_LIB_ERR_LIBGCRYPT_ERROR (-14)
#define CRYPTO_LIB_ERR_AUTHENTICATION_ERROR (-15)
#define CRYPTO_LIB_ERR_NULL_IV (-16)
#define CRYPTO_LIB_ERR_NULL_ABM (-17)
#define CRYPTO_LIB_ERR_DECRYPT_ERROR (-18)
#define CRYPTO_LIB_ERR_ABM_TOO_SHORT_FOR_AAD (-19)
#define CRYPTO_LIB_ERR_MAC_RETRIEVAL_ERROR (-20)
#define CRYPTO_LIB_ERR_MAC_VALIDATION_ERROR (-21)
#define CRYPTO_LIB_ERR_INVALID_HEADER (-22)
#define CRYPTO_LIB_ERR_IV_OUTSIDE_WINDOW (-23)
#define CRYPTO_LIB_ERR_NULL_ARSN (-24)
#define CRYPTO_LIB_ERR_NULL_SA (-25)
#define CRYPTO_LIB_ERR_UNSUPPORTED_ACS (-26)
#define CRYPTO_LIB_ERR_ENCRYPTION_ERROR (-27)
#define CRYPTO_LIB_ERR_INVALID_SA_CONFIGURATION (-28)
#define CRYPTO_LIB_ERR_TC_FRAME_SIZE_EXCEEDS_MANAGED_PARAM_MAX_LIMIT (-29)
#define CRYPTO_LIB_ERR_TC_FRAME_SIZE_EXCEEDS_SPEC_LIMIT (-30)
#define CRYPTO_LIB_ERR_UNSUPPORTED_ECS (-31)
#define CRYPTO_LIB_ERR_KEY_LENGTH_ERROR (-32)
#define CRYPTO_LIB_ERR_NULL_ECS_PTR (-33)
#define CRYPTO_LIB_ERR_IV_NOT_SUPPORTED_FOR_ACS_ALGO (-34)
#define CRYPTO_LIB_ERR_NULL_CIPHERS (-35)
#define CRYPTO_LIB_ERR_NO_ECS_SET_FOR_ENCRYPTION_MODE (-36)
#define CRYPTO_LIB_ERR_IV_LEN_SHORTER_THAN_SEC_HEADER_LENGTH (-37)
#define CRYPTO_LIB_ERR_ARSN_LEN_SHORTER_THAN_SEC_HEADER_LENGTH (-38)
#define CRYPTO_LIB_ERR_FRAME_COUNTER_DOESNT_MATCH_SA (-39)

```

```
#define CRYPTO_LIB_ERR_INPUT_FRAME_TOO_SHORT_FOR_TC_STANDARD (-40)
#define CRYPTO_LIB_ERR_INPUT_FRAME_LENGTH_SHORTER_THAN_FRAME_HEADERS_LENGTH (-41)
#define CRYPTO_LIB_ERR_UNSUPPORTED_ECS_MODE (-42)
#define CRYPTO_LIB_ERR_NULL_MODE_PTR (-43)
#define CRYPTO_LIB_ERR_UNSUPPORTED_MODE (-44)
```

4.2 DCS SDLS HTTPS API Interfaces

The DCS SDLS HTTPS API provides a simple HTTPS interface for performing SDLS operations on telecommand transfer frames (TC TFs). This service is an optional component of the DCS Crypto Service, and supports the following endpoints.

4.2.1 Apply Security

An SDLS HTTPS API for applying SDLS to a TC TF.

Endpoint: `/sdls-service/apply_security`

HTTP Method: POST

Payload: Binary Transfer Frame (application/octet-stream)

Returns: Binary Transfer Frame with SDLS added

Successful execution returns status code 200. Error conditions will return a status code 500, including an error message provided by CryptoLib. See Section 4.1.3.8 for the full list of return messages.

4.2.2 Process Security

An SDLS HTTPS API for processing a TC TF that has SDLS applied to it.

Endpoint: `/sdls-service/process_security`

HTTP Method: POST

Payload: Binary Transfer Frame with SDLS

Returns: JSON Object with all transfer frame fields broken down by key/value. Binary fields are returned as hexadecimal strings.

Example endpoint return:

```
{ "tfvn":0, "bypass":1, "cc":0, "spare":0, "scid":3, "vcid":0, "fl":23, "fsn":0, "sh":0, "spi":1, "iv":"","iv_field_len":0, "sn":"0000", "sn_field_len":2, "pad":"","pad_field_len":0, "tc_pdu":"80D2C70008197F0B00310000", "tc_pdu_len":12, "mac":"","mac_field_len":0, "fecf":48000 }
```

Response Fields:

Field Name	Description
tfvn	Transfer Frame Version Number
bypass	Sequence Check Bypass
cc	Control Command
spare	Reserved
scid	Spacecraft ID
vcid	Virtual Channel ID
fl	Transfer Frame Length
fsn	Frame Sequence Number
sh	Segment Header
spi	Security Parameter Index
iv	Initialization Vector
iv_field_len	Initialization Vector Length
sn	Sequence Number for anti-replay
sn_field_len	Sequence Number length
pad	Count of the used fill bytes
pad_field_len	Pad count length
tc_pdu	Payload Data
tc_pdu_len	Payload Data Length
mac	Message Authentication Code
mac_field_len	Message Authentication Code Length
fecf	Frame Error Control Field

Successful execution returns status code 200. Error conditions will return a status code 500, including an error message provided by CryptoLib. See Section 4.1.3.6 for the full list of return messages.

4.2.3 Status

A simple status endpoint that reports the availability of the SDLS HTTPS API.

Endpoint: `/sdls-service/status`

HTTP Method: GET

Returns: Successful execution returns status code 200. Error conditions will return a status code 500, including an error message provided by CryptoLib. See Section 4.1.3.8 for the full list of return messages.

4.3 Security Association Management Command Line Interface (CLI)

The DCS Security Association (SA) Management (Mgmt) Command Line Interface (CLI) provides the capabilities for creating, updating, listing, deleting, keying/rekeying, starting, stopping, and expiring SAs used for command encryption.

The `kmc-sa-mgmt` CLI provides subcommands for several SA management functions, as shown in the tool's help output shown below.

```
$ /opt/amos/kmc/bin/kmc-sa-mgmt --help

Usage: kmc-sa-mgmt [-hV] [COMMAND]
KMC Security Association Management CLI
  -h, --help          Show this help message and exit.
  -V, --version       Print version information and exit.

Commands:
  list      List security associations
  create    Create a new Security Association
  update    Update an existing Security Association
  delete    Delete a Security Association
  key       Key or rekey a Security Association
  start     Start a Security Association. Only one (1) SA can be active per (SPI,
           GVCID) permutation
  stop      Stop a Security Association
  expire    Expire a Security Association
```

The `kmc-sa-mgmt` CLI can be used to list SAs in the SADB, as described in the usage statement shown below.

```
$ /opt/amos/kmc/bin/kmc-sa-mgmt list --help

Usage: kmc-sa-mgmt list [-hV] [--scid=<scid>] [--spi=<spi>] [-e | --mysql |
           --json] [--active | --inactive]
List security associations
  --active          Only list active SAs
  -e, --extended    extended CSV format
  -h, --help        Show this help message and exit.
  --inactive        Only list inactive SAs
  --json            JSON format
  --mysql            MySQL-like key-value format
  --scid=<scid>     Filter by spacecraft ID
  --spi=<spi>        Filter by security parameter index
  -V, --version     Print version information and exit.
```

The `kmc-sa-mgmt` CLI can be used to create SAs in the SADB, as described in the usage statement shown below.

```
$ /opt/amos/kmc/bin/kmc-sa-mgmt create --help

Usage: kmc-sa-mgmt create [-hV] ([[--tfvn=<tfvn> --scid=<scid> --vcid=<vcid>
           --mapid=<mapid> [--spi=<spi>]] [--ekid=<ekid>]
           [--ecs=<ecs>] [--akid=<akid>] [--acs=<acs>]
           [--iv=<iv>] [--ivlen=<ivlen>] [--arsn=<arsn>]
           [--arsnlen=<arsnlen>] [--arsnw=<arsnw>] [--abm=<abm>]
           [--abmlen=<abmlen>] [--shivflen=<shivflen>]
           [--shsnflen=<shsnflen>] [--shplflen=<shplflen>]
           [--stmacflen=<stmacflen>] [--st=<st>]]] |
```

```

                                [--file=<file>])
Create a new Security Association
--abm=<abm>                    authentication bitmask (hex string)
--abmlen=<abmlen>              authentication bitmask length (bytes)
--acs=<acs>                    authentication cypher suite
--akid=<akid>                  authentication key ID
--arsn=<arsn>                  anti-replay sequence number starting value (hex
                                string)
--arsnlen=<arsnlen>            anti-replay sequence number length (bytes)
--arsnw=<arsnw>                anti-replay sequence number window
--ecs=<ecs>                    encryption cypher suite (hex string)
--ekid=<ekid>                  encryption key ID
--file=<file>
-h, --help                    Show this help message and exit.
--iv=<iv>                      initialization vector (hex string)
--ivlen=<ivlen>                initialization vector length, in bytes
--mapid=<mapid>                multiplexer access ID
--scid=<scid>                  spacecraft ID
--shivflen=<shivflen>          security header initialization vector field length
                                (bytes)
--shplflen=<shplflen>          security header pad length field length (bytes)
--shsnflen=<shsnflen>          security header sequence number field length (bytes)
--spi=<spi>                    security parameter index
--st=<st>                      service type. 0 = PLAINTEXT, 1 = ENCRYPTION, 2 =
                                AUTHENTICATION, 3 = AUTHENTICATED_ENCRYPTION
--stmacflen=<stmacflen>        security trailer MAC field length (bytes)
--tfvn=<tfvn>                  transfer frame version number
-V, --version                  Print version information and exit.
--vcid=<vcid>                  virtual channel ID

```

The kmc-sa-mgmt CLI can be used to update SAs in the SADB. When updating an SA, only the parameters that are being updated need to be specified. If a CSV file is used as input for bulk updates, existing SAs will be updated but new SAs will not be created (use the "create" subcommand to create new SAs).

The usage statement for updating SAs is shown below.

```

$ /opt/amos/kmc/bin/kmc-sa-mgmt update --help

Usage: kmc-sa-mgmt update [-hV] ([[--tfvn=<tfvn>] --scid=<scid>
                                [--vcid=<vcid>] [--mapid=<mapid>] --spi=<spi>]
                                [[--ekid=<ekid>] [--ecs=<ecs>] [--akid=<akid>]
                                [--acs=<acs>] [--iv=<iv>] [--ivlen=<ivlen>]
                                [--arsn=<arsn>] [--arsnlen=<arsnlen>]
                                [--arsnw=<arsnw>] [--abm=<abm>] [--abmlen=<abmlen>]
                                [--shivflen=<shivflen>] [--shsnflen=<shsnflen>]
                                [--shplflen=<shplflen>] [--stmacflen=<stmacflen>]
                                [--st=<st>]]] | [--file=<file>])

Update an existing Security Association
--abm=<abm>                    authentication bitmask (hex string)
--abmlen=<abmlen>              authentication bitmask length (bytes)
--acs=<acs>                    authentication cypher suite
--akid=<akid>                  authentication key ID

```

```

--arsn=<arsn>          anti-replay sequence number starting value (hex
                        string)
--arsnlen=<arsnlen>    anti-replay sequence number length (bytes)
--arsnw=<arsnw>        anti-replay sequence number window
--ecs=<ecs>            encryption cypher suite (hex string)
--ekid=<ekid>          encryption key ID
--file=<file>
-h, --help            Show this help message and exit.
--iv=<iv>             initialization vector (hex string)
--ivlen=<ivlen>        initialization vector length, in bytes
--mapid=<mapid>        multiplexer access ID
--scid=<scid>          spacecraft ID
--shivflen=<shivflen> security header initialization vector field length
                        (bytes)
--shplflen=<shplflen> security header pad length field length (bytes)
--shsnflen=<shsnflen> security header sequence number field length (bytes)
--spi=<spi>            security parameter index
--st=<st>              service type. 0 = PLAINTEXT, 1 = ENCRYPTION, 2 =
                        AUTHENTICATION, 3 = AUTHENTICATED_ENCRYPTION
--stmacflen=<stmacflen>
                        security trailer MAC field length (bytes)
--tfvn=<tfvn>          transfer frame version number
-V, --version          Print version information and exit.
--vcid=<vcid>          virtual channel ID

```

For example, to reset the ICV to a value of "...01" in SPI 123 for SCID 44, use a command like following (with the correct values for your system):

```

$ /opt/ammos/kmc/bin/kmc-sa-mgmt update --spi=123 --scid=44 \
  --icv= X'0000000000000000000000000001'

```

Note: Both the SPI and SCID must be provided because DCS can support using the same SPI for different SCIDs.

The `kmc-sa-mgmt` CLI can be used to delete SAs, as shown in the usage statement below.

```

$ /opt/ammos/kmc/bin/kmc-sa-mgmt delete --help

Usage: kmc-sa-mgmt delete [-hVy] --scid=<scid> --spi=<spi> [--spi=<spi>]...
Delete a Security Association
-h, --help            Show this help message and exit.
--scid=<scid>         spacecraft id
--spi=<spi>            security parameter index
-V, --version          Print version information and exit.
-y                    Delete without confirmation

```

The `kmc-sa-mgmt` CLI can be used to set keys for SAs, as shown in the usage statement below.

```

$ /opt/ammos/kmc/bin/kmc-sa-mgmt key --help

Usage: kmc-sa-mgmt key [-hV] --scid=<scid> --spi=<spi> [--ekid=<ekid>]
                        [--ecs=<ecs>] [--akid=<akid>] [--acs=<acs>] [-y]]
Key or rekey a Security Association

```

```

--acs=<acs>      authentication cypher suite
--akid=<akid>     authentication key ID
--ecs=<ecs>       encryption cypher suite (hex string)
--ekid=<ekid>     encryption key ID
-h, --help       Show this help message and exit.
--scid=<scid>    spacecraft id
--spi=<spi>       security parameter index
-V, --version    Print version information and exit.
-y              Rekey without confirmation

```

The kmc-sa-mgmt CLI can be used to start an SAs, as shown in the usage statement below.

```

$ /opt/amos/kmc/bin/kmc-sa-mgmt start --help
Usage: kmc-sa-mgmt start [-fhV] --scid=<scid> --spi=<spi> [--spi=<spi>]...
Start a Security Association. Only one (1) SA can be active per (SPI, GVCID)
permutation
-f, --force      Force an SA to start, if another SA is currently active.
-h, --help       Show this help message and exit.
--scid=<scid>    spacecraft ID
--spi=<spi>       security parameter index
-V, --version    Print version information and exit.

```

The kmc-sa-mgmt CLI can be used to stop an SA, as shown in the usage statement below.

```

$ /opt/amos/kmc/bin/kmc-sa-mgmt stop --help
Usage: kmc-sa-mgmt stop [-fhV] --scid=<scid> --spi=<spi> [--spi=<spi>]...
Stop a Security Association
-f, --force      Force an SA to start, if another SA is currently active.
-h, --help       Show this help message and exit.
--scid=<scid>    spacecraft ID
--spi=<spi>       security parameter index
-V, --version    Print version information and exit.

```

The kmc-sa-mgmt CLI can be used to expire an SA, as shown in the usage statement below.

```

$ /opt/amos/kmc/bin/kmc-sa-mgmt expire --help
Usage: kmc-sa-mgmt expire [-hVy] --scid=<scid> --spi=<spi> [--spi=<spi>]...
Expire a Security Association
-h, --help       Show this help message and exit.
--scid=<scid>    Spacecraft id
--spi=<spi>       Security parameter index
-V, --version    Print version information and exit.
-y              Expire without confirmation

```

Expiring an SA will clear the ekid/akid (i.e., the key reference) and set sa_state to 1 (unkeyed). To re-use an SA that has been expired, use the "key" subcommand to set the key, and then the "start" subcommand (when ready to start the SA).

Section 5

DCS Secure Connection Interfaces

5.1 DCS Secure Connections

Warning – system security depends on the security of SSO Tokens, cryptographic keys, and other security sensitive information sent over network connections. They should be protected from unauthorized access, particularly in-transit. Use HTTPS (or other appropriate secure protocol) to protect SSO Tokens (and other important information) sent over network connections.

5.1.1 Java Truststore

Transport Layer Security (TLS) connections between the DCS software libraries and the Crypto Service or KMS rely on TLS keys, digital certificates, and a Java truststore. The DCS Product Guide describes how to deploy those files.

The Java truststore required by the DCS Crypto Java libraries contains the certificates of the Certificate Authority (CA) that issue the TLS server certificates to the Crypto Service and/or KMS (depending on which of those two services are used). Alternatively (but not typical), the truststore can contain the certificates of the Crypto Service and KMS instead of the CAs.

If the KMS and Crypto Service use certificates from a well-known CA (i.e., whose CA certificates are included in the default Java truststore), then the default Java truststore may be used. Otherwise, the DCS Crypto Java Libraries need a special truststore. The application is responsible for setting the Java truststore system property if the default Java truststore is not used. Generally, AMMOS software applications (written in Java) use the following truststore: `/ammos/etc/pki/tls/certs/ammos-truststore.jks`.

5.1.2 Key Management Service Mutual Authentication

The KMIP requires mutual authentication at TLS layer between the KMS client (i.e., the Crypto Service and a Crypto Client that will use a KMS) and the KMS.

When TLS mutual authentication is used, the client must authenticate to the server (in addition to the server authenticating to the client). Thus, the client needs a TLS key and certificate.

Instructions for setting up TLS mutual authentication are provided in the DCS Product Guide. See the DCS Product Guide for the version of DCS that is used in your venue.

5.1.3 Key Management Service Connection Timeout

If the KMS does not respond to a client KMIP request, the default TCP connection timeout is 15 minutes, which is too long for normal operations. The DCS Crypto Client sets two default timeouts:

- **Connection Timeout:** The connection timeout is the timeout in making the initial connection; i.e. completing the TCP connection handshake. The DCS default value is 10 seconds.
- **Read Timeout:** The read timeout is the timeout on waiting to read data. Specifically, if the server fails to send a byte <timeout> seconds after the last byte, a read timeout error will be raised. The DCS default value is 30 seconds.

The default timeout values can be overridden with the following parameters in the DCS Crypto configuration file (showing acceptable values):

`connection_timeout_connect=2000`

`connection_timeout_read=5000`

See the DCS Product Guide (for the version being used) for instructions for customizing the timeout values.

Appendix A: Acronyms and Abbreviations

3DES	Triple-DES
AE	Authenticated Encryption
AEAD	Authenticated Encryption with Associated Data
AES	Advanced Encryption Algorithm
AMMOS	Advanced Multi-Mission Operations System
API	Application Programming Interface
App	Application
CA	Certificate Authority
CAM	Common Access Manager
CCSDS	Consultative Committee for Space Data Systems
CBC	Cipher Block Chaining
CLI	Command Line Interface
CMAC	Cipher-Based Message Authentication Code
DCS	Data Cryptography Services
DES	Data Encryption Standard
DESede	DES Encryption Decryption Encryption (a.k.a., Triple-DES)
DSN	Deep Space Network
ECB	Electronic Code Book
ESA	European Space Agency
FECF	Frame Error Control Field
GCM	Galois/Counter Mode
GUI	Graphical User Interface
HMAC	Hash-based Message Authentication Code
HTTP	Hyper-Text Transfer Protocol
HTTPS	Secure Hyper-Text Transfer Protocol
ICV	Integrity Check Value
IV	Initialization Vector
JPL	Jet Propulsion Laboratory

KMC	Key Management and Cryptography
KMIP	Key Management Interoperability Protocol
MAC	Message Authentication Code
Mgmt	Management
MGSS	Multimission Ground Systems & Services
PKCS	Public-Key Cryptography Standards
NASA	National Aeronautics and Space Administration
RSA	Rivest, Shamir, Adelman
SDLS	Space Data Link Security
SHA	Secure Hash Algorithm
SMP	Software Management Plan
SSL	Secure Sockets Layer
SSO	Single Sign-On
TLS	Transport Layer Security
URI	Universal Resource Identifier
URL	Universal Resource Locator

Appendix B: DCS Metadata

The DCS Metadata is a string of attribute name:value pairs, with a colon (:) to separate the attribute name and value. The pairs in the metadata are unordered and separated by commas.

In general, the metadata does not have to be parsed or constructed by applications that use DCS. The information about the metadata is provided here in support of applications that need to convert between DCS metadata and some other representation of that metadata (e.g., as done internally within DCS between the data-level cryptography and command encryption capabilities).

Integrity Metadata

Integrity metadata has these attributes:

1. `metadataType: IntegrityCheckMetadata`
This attribute identifies the type of the metadata.
2. `keyRef`
This attribute provides a reference to the key to be used for generating the ICV.
3. `cryptoAlgorithm`
This attribute provides the name of the algorithm to use.
4. `integrityCheckValue`
This attribute provides the Integrity Check Value encoded in URL-Safe Base 64
5. `macLength`
This attribute specifies the length of the MAC as integer for number of bits.

Examples:

1) AES-CMAC example without `macLength`:

`metadataType: IntegrityCheckMetadata, keyRef: kmc/test/nist_cmac_95, cryptoAlgorithm: AESCMAC, integrityCheckValue: A_WS2BzlAxMxKa0RkXq24Q==`

2) AES-CMAC example with `macLength` for truncated (64-bit) MAC:

`metadataType: IntegrityCheckMetadata, keyRef: kmc/test/nist_cmac_95, cryptoAlgorithm: AESCMAC, integrityCheckValue: A_WS2BzlAxMxKa0RkXq24Q==, macLength: 64`

3) Digital signature example with RSA-SHA512:

`metadataType: IntegrityCheckMetadata, keyRef: kmc/test/RSA4096, cryptoAlgorithm: SHA512withRSA, integrityCheckValue: u6HS2EWhcYzQbPsT8PIO0spFEwfrJN2p-mm_ZvBWSAU1LkGzNkbyJBL_mid1z-f2D3yaO-S-4ssqB3cvsetgs9OmegydpIAVxxwhyY4DPhANpOnpwkHE1SYcFanvo18h__WaVZ_wKhrVr0eiaTmvn02rMiCHGbkaes07tU6z8wFw71eBgnPomT6DJeSdETW4_v259s776SUcgqZ4OG3PueltAf2yRx11-ITqM-nBeosNXnRwver_hK_YNv1f2L24i4K_ITjsZ0y1KsEivESy29zCVXFkibQxspfxTe3ylgRnlnPgjMYKXUg3z3HWygchQneXc2oM4Xpu34us82DuTin1une2isnIxs4lwVxPo614TTy52BV1WWisagUL2QTNIKPFqLFYDZ6O49WgHx5pusdbAAO5RZ50hg6LV2HlsUoHRCUGBxfiKte664ESxVaiVc32Z54IXhfMpbUzipj0ZURHoqKp1hMKJ6yuCVWxzgp1V8Nq94Qn2JT3BVFcmjZpJvVHW-bX2JR3x_0PMZEEQXs9nn4S-D7CFCPyjIJEiDSpEtg7xn878IaZD5hm9kHJSLCMgs7dBFIFVhaZVMiSAsMplcEqgssgWvFZ5MMTB8bqG52-eQQZajvMBF48mZcT2dA9rESYvvoZWS0GleauKE_AgGOX7urXNOrq7EaOs=,`

Encryption Metadata

Encryption metadata has these attributes:

1. metadataType:EncryptionMetadata
This attribute identifies the type of the metadata.
2. keyRef
This attribute provides a reference to the key to be used.
3. keyLength
This attribute provides the length of the key as integer for number of bits.
4. cryptoAlgorithm
This attribute provides the name of the algorithm to use.
5. cipherTransformation
This attribute provides the name of the cipher transformation to be used.
6. initialVector (encoded in URL-Safe Base64)
This attribute provides the InitialVector
7. encryptOffset (optional)
This attribute provides the byte from which encryption is applied.
8. macLength (optional)
This attribute specifies the length of the MAC as integer for number of bits.

Examples:

1) AES-GCM Example without macLength:

keyLength:256,keyRef:kmc/test/AES256,initialVector:UHeYcyshrdMg9vuH,cryptoAlgorithm:AES,
cipherTransformation:AES/GCM/NoPadding, encryptOffset:16

2) AES-GCM Example with macLength for truncated (64-bit) MAC:

keyLength:256,keyRef:kmc/test/AES256,initialVector:UHeYcyshrdMg9vuH,cryptoAlgorithm:AES,
cipherTransformation:AES/GCM/NoPadding,macLength:64,encryptOffset:16

3) AES-CBC example:

metadataType:EncryptionMetadata, cryptoAlgorithm:AES,cipherTransformation:
AES/CBC/PKCS5Padding,keyLength:256, keyRef:kmc/test/AES256,initialVector:hkBtDz5jUVfOn1CfgykM-
g==

Appendix C: Example Use of the DCS SDLS Python Interface

Overview

The test application described in this appendix can be used to verify the DCS setup, and as an example application that uses the DCS SDLS Python library.

DCS 3.7 provides Python 3.9 interfaces (wrappers) to the DCS SDLS C interface. DCS SDLS interfaces can be used to apply & process SDLS on TC transfer frames.

The DCS software must be configured and all keys/keystores/certificates/truststores must be in place before the test application can be used. See the DCS 3.7 Product Guide for deployment instructions.

DCS SDLS PYTHONPATH

In order to utilize the DCS SDLS Python interfaces, the appropriate location of the DCS SDLS Python library location must be included in the PYTHONPATH environment variable. Set PYTHONPATH before running the test application.

Examples of setting PYTHONPATH (using locations in a standard AMMOS RPM based install) are shown below.

Bourne Shell variants (sh, ksh, bash, zsh, etc):

```
$ export PYTHONPATH=/opt/ammios/kmc/lib/python3.9/site-packages
```

C-Shell variants (csh, tcsh, etc):

```
$ setenv PYTHONPATH /opt/ammios/kmc/lib/python3.9/site-packages
```

Note that PYTHONPATH must be set according to the python version that will be used. The default python3 version can be checked by using the following command:

```
$ /usr/bin/python -version
Python 3.9.21
```

DCS SDLS Test Application

A simple test application is included in the DCS Crypto Client. The test application provides the ability to use the SDLS ApplySecurity and/or ProcessSecurity capabilities of DCS.

The test application can be found at the following location:

- /opt/ammios/kmc/test/kmc_sdls_test_app.py

The -h option provides usage info, as shown below.

```
$ ./kmc_sdls_test_app.py -h
```

```
usage: kmc_sdls_test_app.py [-h] [-f FRAME] [-p PROPERTIES] [-P] [-A]
Simple KMC SDLS Python Test Application that will Apply and Process Security
on a given frame
optional arguments:
  -h, --help            show this help message and exit
```

```

-f FRAME, --frame FRAME
                                Hex frame string representation of telecommand
                                transfer-frame to apply & process SDLs layering on.
                                Defaults to: '202c0408000001bd37'

-p PROPERTIES, --properties PROPERTIES
                                The properties file that contains all the
                                configuration needed by this application (supported
                                properties defined in KMC SIS)

-P, --processOnly               Flag to only process security on the frame (default is
                                to apply & process)

-A, --applyOnly                 Flag to only apply security on the frame (default is
                                to apply & process)

```

The test application reads in configuration parameters from the following file by default, or a different configuration file can be specified by using the `-p` (or `--properties`) option:

- `/opt/ammos/kmc/test/kmc_sdls_test_app.properties`

To use the test application, set the parameters in the configuration file to the correct values. At a minimum, set the following parameters:

- `cryptolib.sadb.mariadb.fqdn`
 - Fully-qualified domain name of the mariadb (for SADB) server host
- `cryptolib.sadb.mariadb.username`
 - Username of account in mariadb for accessing the SADB
- `cryptolib.sadb.mariadb.mtls.clientcert`
 - Client certificate for account in mariadb for accessing the SADB
 - This should be the same client-side certificate that is used by users of the command and control application that uses DCS for command encryption
 - Can use the `cryptolib.sadb.mariadb.password` parameter instead (if that is how the account is set up in mariadb), but that is not the recommended configuration
- `cryptolib.sadb.mariadb.mtls.clientkey`
 - Key file for account in mariadb for accessing the SADB
 - Must match the client-side certificate that is being used
 - Can use the `cryptolib.sadb.mariadb.password` parameter instead (if that is how the account is set up in mariadb), but that is not the recommended configuration
- `cryptolib.crypto.kmccryptoservice.fqdn`
 - Fully-qualified domain name of the DCS Crypto Service server host

- `cryptolib.crypto.kmccryptoservice.mtls.clientcert`
 - Client certificate file for accessing the DCS Crypto Service
 - This should be the same client-side certificate that is used by users of the command and control application that uses DCS for command encryption
 - This can be the same certificate that is used for accessing the SADB (which is how the instructions in the DCS 3.7 Product Guide set up the system), depending on how the services are set up -- but a different certificate could be used
- `cryptolib.crypto.kmccryptoservice.mtls.clientkey`
 - Client key file for accessing the DCS Crypto Service
 - Must match the client-side certificate that is being used

By default, the test application uses a built-in input frame for SCID 44. The `-f` (or `--frame`) option can be used to supply a different test frame on the command line. Additional settings may need to be edited in the configuration file, depending on specifics of the test frame (and setup -- such as using a different key/certificate file format). The default settings for the frame-related parameters in the delivered `kmc_sdls_test_app.properties` file align with the built-in test frame. See the configuration file for the full set of parameters, which are also described in Section 4.1.1.1.1 of this document.